FOODRESCUE

Hệ thống web giải cứu thực phẩm cuối ngày

Tài liệu phân tích, thiết kế và kế hoạch dự án

Môn học: Lập trình J2EE

Backend: Spring Boot 3 (REST API) · Frontend: ReactJS · CSDL: MySQL 8

Tích hợp: VNPay Sandbox · Bản đồ OpenStreetMap · OSRM (tính khoảng cách)

4 nhóm người dùng: Quản trị viên · Chủ cửa hàng · Nhân viên cửa hàng · Khách hàng

Phiên bản 5.0 (cấu trúc 3 lớp Controller – Service – Repository, kế hoạch 5 tuần) · Ngày 17/10/2026

Mục lục

1. Tổng quan dự án

1.1 Bối cảnh · 1.2 Giải pháp · 1.3 Mục tiêu · 1.4 Phạm vi

2. Người dùng và phân quyền

3. Chức năng hệ thống

3.1 Khách hàng · 3.2 Nhân viên · 3.3 Chủ cửa hàng · 3.4 Quản trị viên · 3.5 Tác vụ tự động

4. Quy tắc nghiệp vụ

5. Trạng thái, quy trình và dòng tiền

5.5 Hoàn tiền · 5.6 Chi trả theo kỳ · 5.7 Đối soát · 5.8 Sao kê · 5.9 Nhật ký · 5.10 Đối chiếu nghiệp vụ và CSDL

6. Tích hợp API bên ngoài

6.1 VNPay Sandbox · 6.2 Bản đồ và tính khoảng cách

7. Kiến trúc và công nghệ

7.1 Kiến trúc · 7.2 Công nghệ · 7.3 Đăng nhập JWT · 7.4 Chống bán vượt · 7.5 Cấu trúc thư mục và chức năng từng file · 7.6 Chạy dự án

8. Danh sách API

9. Danh sách màn hình

10. Thiết kế cơ sở dữ liệu

11. Kế hoạch thực hiện

12. Rủi ro

13. Kiểm thử

14. Tài khoản và kịch bản demo

Phụ lục A. Script cơ sở dữ liệu

Phụ lục B. Câu truy vấn kiểm tra dữ liệu

1. Tổng quan dự án

1.1 Bối cảnh

Cuối ngày, tiệm bánh, quán ăn, cửa hàng tiện lợi thường còn dư thức ăn vẫn đảm bảo chất lượng nhưng không bán sang hôm sau được, phần lớn bị bỏ đi. Trong khi đó nhiều người (sinh viên, người đi làm về muộn) sẵn sàng mua với giá rẻ nếu biết chỗ nào gần mình đang còn.

1.2 Giải pháp

FoodRescue là website trung gian. Cửa hàng đăng **"túi giải cứu"** với giá giảm 40-70% kèm khung giờ đến lấy. Khách xem các túi gần mình (có khoảng cách thực tế trên bản đồ), đặt và thanh toán qua **VNPay**, rồi đến quầy đọc **mã nhận hàng**. Nhân viên nhập mã để hoàn tất đơn. Nền tảng giữ lại 15% phí và chuyển phần còn lại cho cửa hàng.

1.3 Mục tiêu

- Website chạy ổn định, đủ 4 vai trò, đi trọn luồng: **đăng túi → tìm theo khoảng cách → thêm vào giỏ → đặt → thanh toán VNPay Sandbox → nhận hàng bằng mã → đánh giá → admin chi trả cho cửa hàng**.
- Không bán vượt số lượng khi nhiều người cùng đặt một túi.
- **Tiền rõ ràng:** mọi đồng khách trả đều biết đang ở đâu (đã hoàn, đã chi cho cửa hàng, phí nền tảng, hay đang giữ); đối soát khớp tới từng đồng; mọi thao tác tiền đều có nhật ký.
- Hoàn thành trong 5 tuần, code chạy được trên máy cá nhân và có thể deploy lên mạng để demo.

1.4 Phạm vi

Vì đồ án chiếm 25% điểm, phạm vi được cắt về **những gì cần để website vận hành được**. Các phần phụ được liệt kê để nhóm có thể nêu là "hướng phát triển" khi báo cáo.

Giữ lại (bắt buộc làm)

- Đăng ký / đăng nhập (JWT), 4 vai trò
- Cửa hàng đăng ký, admin duyệt
- Đăng / sửa / huỷ túi giải cứu, thêm / bớt túi, **sổ kho**
- Tìm túi, bản đồ, **khoảng cách thực tế (OSRM)**
- **Giỏ hàng** (nhiều túi của 1 cửa hàng), đặt hàng, **thanh toán VNPay Sandbox**
- Mã nhận hàng (hiện kèm QR), nhân viên nhập mã giao hàng
- Cửa hàng huỷ đơn khi hết hàng; **ghi nhận hoàn tiền** cho từng đơn
- Đánh giá sao sau khi nhận hàng
- Chủ cửa hàng: tài khoản nhân viên, doanh thu, **sao kê từng đơn**
- Admin: duyệt cửa hàng, khoá người dùng, danh mục, đơn hàng, **chi trả theo kỳ, đối soát, nhật ký thao tác**, thống kê
- 2 tác vụ tự động: huỷ đơn chưa thanh toán, hết hạn túi / ghi nhận không đến lấy

Đã lược bỏ (hướng phát triển)

- Xác thực email, quên mật khẩu qua email, gửi email (thay bằng admin đặt lại mật khẩu tạm)
- Refresh token
- Khách tự huỷ đơn sau khi đã thanh toán
- Gọi API hoàn tiền VNPay tự động (admin hoàn thủ công)
- Cửa hàng bấm xác nhận đơn (thanh toán xong là xác nhận luôn)
- Khiếu nại, yêu thích, thông báo trong web
- WebSocket realtime (thay bằng tự tải lại mỗi 15 giây)
- Ví tạm giữ, cửa hàng tự yêu cầu rút tiền (thay bằng admin chi trả theo kỳ)
- Phân quyền chi tiết cho nhân viên, phân ca
- Geocoding (chủ quán tự ghim vị trí trên bản đồ)
- Voucher, cấu hình động, Cloudinary
- Khoá tài khoản khách hay bom hàng

2. Người dùng và phân quyền

Vai trò	Làm được gì
Khách hàng `CUSTOMER`	Xem và tìm túi gần mình, thêm vào giỏ, đặt hàng, thanh toán VNPay, xem mã nhận hàng, đánh giá. Chưa đăng nhập vẫn xem được danh sách và bản đồ.
Nhân viên `STORE_STAFF`	Làm việc tại quầy của 1 cửa hàng: xem đơn hôm nay, nhập mã để giao hàng, thêm / bớt túi, xem sổ kho.
Chủ cửa hàng `STORE_OWNER`	Mọi việc của nhân viên, cộng: hồ sơ cửa hàng, đăng/sửa/huỷ túi, huỷ đơn, tạo/khoá tài khoản nhân viên, xem doanh thu và số tiền được nhận.
Quản trị viên `ADMIN`	Duyệt/khoá cửa hàng, khoá người dùng, quản lý danh mục, xem mọi đơn, xác nhận hoàn tiền, tạo và xác nhận kỳ chi trả cho cửa hàng, đối soát tiền, xem nhật ký thao tác, xem thống kê.

Chức năng	Khách	Nhân viên	Chủ CH	Admin
Xem túi, bản đồ, khoảng cách	✓	✓	✓	✓
Giỏ hàng, đặt hàng, thanh toán, đánh giá	✓			
Xem đơn cửa hàng, nhập mã giao hàng, thêm / bớt túi, xem sổ kho		✓	✓	
Đăng/sửa/huỷ túi, huỷ đơn			✓	
Tài khoản nhân viên, doanh thu			✓	
Duyệt cửa hàng, khoá người dùng, danh mục, hoàn tiền, chi trả, đối soát, nhật ký, thống kê				✓

Phân quyền kiểm tra ở backend bằng Spring Security (`@PreAuthorize` theo vai trò). Chủ cửa hàng và nhân viên chỉ thao tác được trên dữ liệu thuộc cửa hàng của mình.

3. Chức năng hệ thống

3.1 Khách hàng (và khách chưa đăng nhập)

Mã	Chức năng	Mô tả
C-01	Đăng ký, đăng nhập	Đăng ký bằng email, mật khẩu, họ tên, SĐT. Đăng nhập chung cho mọi vai trò, sau đó React chuyển tới trang của vai trò đó. Tài khoản đang dùng mật khẩu tạm (nhân viên mới, vừa được admin đặt lại) bị chuyển tới trang đổi mật khẩu trước.
C-02	Trang chủ, danh sách túi	Hiện túi đang bán; lọc theo danh mục, bán kính (1/3/5/10 km); sắp xếp gần nhất / rẻ nhất / sắp hết giờ. Mỗi túi hiện "2,3 km · 8 phút".
C-03	Bản đồ quanh tôi	Lấy vị trí bằng trình duyệt (hoặc bấm chọn trên bản đồ); hiện marker các cửa hàng có túi đang bán.
C-04	Chi tiết túi	Ảnh, mô tả, giá gốc / giá giải cứu, số còn lại, khung giờ lấy, địa chỉ, khoảng cách, điểm đánh giá cửa hàng; chọn số lượng và nút **Thêm vào giỏ**.
C-05a	Giỏ hàng	Thêm túi vào giỏ (mỗi loại 1-5 cái), sửa số lượng, xoá. Giỏ chỉ chứa túi của **một cửa hàng**; thêm túi của quán khác thì hỏi "Xoá giỏ hiện tại?". Giỏ lưu trong CSDL (đổi máy vẫn còn) nhưng **không giữ hàng**; mở giỏ thì túi đã hết hoặc hết giờ được đánh dấu và không đặt được.
C-05b	Đặt hàng và thanh toán	Từ giỏ bấm "Đặt hàng": hệ thống kiểm tra lại từng túi, trừ kho **tất cả hoặc không túi nào** (thiếu 1 túi thì báo túi nào thiếu), tính khung giờ lấy chung, tạo đơn + chi tiết đơn, xoá giỏ, chuyển sang VNPay, quay lại trang kết quả. Đơn chưa thanh toán thì khách tự huỷ được (số lượng được trả lại ngay).
C-06	Đơn của tôi	Danh sách đơn và trạng thái; đơn đã thanh toán hiện **mã nhận hàng + QR**, khung giờ lấy, nút "Chỉ đường" (mở Google Maps); mỗi đơn hiện số tiền đã trả; đơn bị huỷ hiện **trạng thái hoàn tiền** (đang hoàn / đã hoàn + mã giao dịch).
C-07	Đánh giá	Sau khi nhận hàng: chấm 1-5 sao và bình luận, mỗi đơn 1 lần.
C-08	Hồ sơ	Sửa họ tên, SĐT, đổi mật khẩu.

3.2 Nhân viên cửa hàng

Mã	Chức năng	Mô tả
ST-01	Màn hình quầy	Đơn hôm nay của cửa hàng, nhóm theo trạng thái; tự tải lại mỗi 15 giây để thấy đơn mới.
ST-02	Giao hàng bằng mã	Nhập mã 6 ký tự khách đọc → hiện tên khách và danh sách túi, số lượng → bấm "Đã giao" → đơn hoàn tất.
ST-03	Thêm / bớt túi	Thêm túi (làm thêm được) hoặc bớt túi (khách tại quầy mua mất). Hệ thống cộng/trừ đồng thời `quantity` và `remaining_quantity` (BR-18); không bớt quá số còn lại.
ST-04	Sổ kho	Xem lịch sử tăng/giảm của từng túi: đăng, thêm, bớt, khách đặt, trả lại; ai làm, lúc nào, còn lại bao nhiêu.

3.3 Chủ cửa hàng

Mã	Chức năng	Mô tả
O-01	Đăng ký cửa hàng	Tạo tài khoản chủ + thông tin cửa hàng, **ghim vị trí trên bản đồ**, giờ mở cửa, tài khoản ngân hàng, ảnh giấy phép VSATTP. Chờ admin duyệt.
O-02	Hồ sơ cửa hàng	Sửa thông tin, ảnh, vị trí, giờ mở cửa, tài khoản ngân hàng.

Mã	Chức năng	Mô tả
O-03	Quản lý túi giải cứu	Tạo / sửa / huỷ túi: tiêu đề, danh mục, ảnh, mô tả, giá gốc, giá giải cứu, số lượng, khung giờ lấy. Túi đã có đơn thì không sửa giá và khung giờ (đơn cũ vẫn giữ giá lúc mua). Mọi lần đăng, thêm, bớt đều ghi vào sổ kho.
O-04	Quản lý đơn	Xem đơn theo ngày và trạng thái; giao hàng bằng mã; **huỷ đơn** khi hết hàng (phải ghi lý do, đơn chuyển sang chờ hoàn tiền); chi tiết đơn liệt kê từng túi, số lượng, giá lúc mua.
O-05	Nhân viên	Tạo tài khoản nhân viên (email + mật khẩu tạm, nhân viên phải đổi ở lần đăng nhập đầu), khoá / mở khoá.
O-06	Doanh thu và sao kê	Theo khoảng ngày: số đơn, số túi bán, doanh thu, phí, thực nhận; biểu đồ theo ngày; **sao kê từng đơn** (đã chi ở kỳ nào / chưa chi); danh sách kỳ chi trả kèm mã chuyển khoản; tổng tiền chưa được chi; xem đánh giá.
O-07	Yêu cầu rút tiền	Bấm "Yêu cầu rút tiền": hệ thống gom mọi đơn chưa chi đến hôm nay thành một kỳ chi trả `PENDING` gửi admin; theo dõi đến khi admin chuyển khoản (có mã giao dịch).

3.4 Quản trị viên

Mã	Chức năng	Mô tả
A-01	Thống kê	Tổng đơn, doanh thu (GMV), phí thu được, số túi đã giải cứu, số cửa hàng / khách hàng; biểu đồ theo ngày.
A-02	Duyệt cửa hàng	Xem thông tin, ảnh giấy phép, vị trí trên bản đồ → duyệt (đặt **tỉ lệ phí**, mặc định 15%, sửa được sau) hoặc từ chối (ghi lý do). Khoá / mở khoá cửa hàng: khi khoá thì túi bị ẩn, còn đơn đã thanh toán vẫn phải được giao hoặc admin huỷ và hoàn tiền.
A-03	Người dùng	Danh sách, tìm theo email/vai trò, khoá / mở khoá; **đặt lại mật khẩu tạm** khi người dùng quên (người dùng bị bắt đổi ở lần đăng nhập sau).
A-04	Danh mục	Thêm / sửa / xoá danh mục túi.
A-05	Đơn hàng và hoàn tiền	Xem mọi đơn, tra theo mã; chi tiết đơn có **lịch sử tiền** (thanh toán, hoàn tiền, kỳ chi trả). Trang Hoàn tiền: chờ hoàn / đã hoàn; admin hoàn trên VNPay Sandbox rồi **nhập mã giao dịch hoàn** để xác nhận (mục 5.5).
A-06	Chi trả theo kỳ	Bảng tiền chưa chi của từng cửa hàng; tạo kỳ chi trả (chọn cửa hàng + khoảng ngày, hệ thống gom đơn và tính tiền); chuyển khoản rồi nhập mã chuyển khoản; xử lý các yêu cầu rút tiền cửa hàng gửi lên; xem chi tiết từng kỳ (mục 5.6).
A-07	Đối soát tổng	Trang đối soát theo khoảng ngày: 7 khoản tiền và dòng kiểm tra chênh lệch = 0 (mục 5.7); bảng theo từng cửa hàng.
A-08	Nhật ký thao tác	Danh sách nhật ký, lọc theo hành động, loại đối tượng, người thực hiện, ngày; bấm vào để mở đơn / kỳ chi trả / cửa hàng liên quan.

3.5 Tác vụ tự động

Tác vụ	Chu kỳ	Việc làm
Huỷ đơn chưa thanh toán	1 phút	Đơn `PENDING_PAYMENT` quá **15 phút** → `EXPIRED`, cộng trả số lượng cho các túi trong đơn (ghi sổ kho `RETURNED`). (Link VNPay hết hạn ở phút 10; chờ thêm 5 phút cho kết quả thanh toán về muộn.)
Hết hạn túi, không đến lấy	5 phút	Túi quá `pickup_end` → `EXPIRED`. Đơn `CONFIRMED` quá `pickup_end` + 30 phút → `NO_SHOW` (không hoàn tiền, cửa hàng vẫn nhận tiền).

4. Quy tắc nghiệp vụ

Mã	Quy tắc
BR-01	Email là duy nhất; mật khẩu ≥ 6 ký tự, lưu mã hoá BCrypt. Tài khoản bị khoá không đăng nhập được. Tài khoản có `must_change_password = TRUE` chỉ gọi được API đổi mật khẩu cho đến khi đổi xong.
BR-02	Cửa hàng chỉ được đăng túi và hiện lên trang khách khi đã được admin duyệt (`APPROVED`). Mỗi tài khoản chủ có đúng 1 cửa hàng; mỗi nhân viên thuộc đúng 1 cửa hàng.
BR-03	Giá giải cứu > 0 và ≤ 60% giá gốc (giảm tối thiểu 40%).
BR-04	Khung giờ lấy: giờ kết thúc sau giờ bắt đầu, và giờ kết thúc chưa qua.
BR-05	Mỗi loại túi trong giỏ / đơn có 1-5 cái; không được bán vượt số còn lại (mục 7.4); đơn nhiều túi thì trừ kho tất cả hoặc không trừ túi nào. Hết hàng thì túi chuyển `SOLD_OUT` .
BR-06	Đặt hàng thì trừ số lượng ngay; link VNPay hết hạn sau **10 phút**; quá **15 phút** chưa thanh toán thì đơn hết hạn và trả số lượng. Khách tự huỷ được đơn chưa thanh toán.
BR-07	Thanh toán thành công (chữ ký VNPay hợp lệ, số tiền khớp) thì đơn chuyển thẳng sang `CONFIRMED` .
BR-08	Khách không tự huỷ đơn đã thanh toán. Chỉ chủ cửa hàng (hết hàng) hoặc admin được huỷ, và chỉ huỷ được đơn `CONFIRMED` (chưa giao). Khi huỷ: tạo 1 bản ghi hoàn tiền bằng đúng `total_amount` , đơn chuyển `REFUND_PENDING` , số lượng được trả lại túi.
BR-09	Chỉ giao hàng khi đơn `CONFIRMED` , đúng cửa hàng, trong khoảng từ 15 phút trước giờ bắt đầu lấy đến 30 phút sau giờ kết thúc.
BR-10	Khách không đến lấy (quá giờ kết thúc 30 phút) thì đơn là `NO_SHOW` , không hoàn tiền.
BR-11	Chỉ đánh giá đơn `COMPLETED` , mỗi đơn 1 lần.
BR-12	Tỉ lệ phí của cửa hàng (`stores.commission_rate` , mặc định 15%, admin đặt) được **copy vào đơn lúc đặt** (`orders.commission_rate`): `commission = round(total × rate / 100)` , `store_earning = total − commission` . Đổi tỉ lệ chỉ áp dụng cho đơn mới.
BR-13	Đơn `COMPLETED` hoặc `NO_SHOW` không huỷ, không hoàn được; số tiền cửa hàng nhận của các đơn này là cố định.
BR-14	Chỉ đơn `COMPLETED` / `NO_SHOW` mới được đưa vào kỳ chi trả; mỗi đơn thuộc tối đa 1 kỳ; `net_amount` của kỳ = tổng `store_earning` của đúng các đơn trong kỳ.
BR-15	Kỳ chi trả chỉ chuyển `PAID` khi có mã chuyển khoản; kỳ đã `PAID` không sửa, không huỷ. Tài khoản ngân hàng nhận tiền được chụp lại vào kỳ lúc tạo.
BR-16	Hoàn tiền chỉ chuyển `COMPLETED` khi có mã giao dịch hoàn; mỗi đơn hoàn tối đa 1 lần.
BR-17	Mọi thao tác tiền và quản trị đều ghi `audit_logs` . Không xoá đơn, thanh toán, hoàn tiền, kỳ chi trả hay nhật ký.
BR-18	`quantity` là tổng số túi đã đưa lên app, `remaining_quantity` là số còn lại. Thêm hoặc bớt túi thì cộng/trừ **cả hai**, để `quantity − remaining_quantity` luôn bằng số túi đã bán qua app.
BR-19	Thanh toán về khi đơn đã hết hạn: nếu túi còn hàng thì trừ lại số lượng và xác nhận đơn; nếu hết thì huỷ đơn và tự tạo yêu cầu hoàn tiền (lý do `LATE_PAYMENT`). Khách không bao giờ mất tiền mà không có hàng.
BR-20	Cửa hàng yêu cầu rút tiền khi có ít nhất 1 đơn chưa chi và không có kỳ nào đang `PENDING` ; kỳ được tạo gồm mọi đơn chưa chi đến hôm nay.
BR-21	Giỏ hàng chỉ chứa túi của 1 cửa hàng, không giữ hàng; giá và số còn lại luôn lấy mới nhất khi mở giỏ và khi đặt. Đặt thành công thì giỏ được xoá.
BR-22	Khung giờ lấy của đơn = phần giao nhau khung giờ các túi trong đơn (bắt đầu muộn nhất đến kết thúc sớm nhất); không có phần giao nhau thì không cho đặt chung.

Mã	Quy tắc
BR-23	Mọi thay đổi số lượng túi (đăng, thêm, bớt, đặt, trả lại) chỉ đi qua một hàm dùng chung và ghi 1 dòng `stock_movements`; tổng các dòng luôn bằng số còn lại.
BR-24	**Huỷ túi** khi còn đơn: đơn `CONFIRMED` có túi đó được huỷ cả đơn và hoàn toàn bộ (`BATCH_CANCELLED`); đơn `PENDING_PAYMENT` có túi đó chuyển `CANCELLED` + `UNPAID` và trả kho; nếu tiền VNPay của đơn đó vẫn về sau thì xử lý như BR-19 (tự hoàn). Túi trong giỏ hàng bị đánh dấu "không còn bán".
BR-25	**Khoá cửa hàng:** túi bị ẩn, không thêm vào giỏ và không đặt được; đơn `CONFIRMED` vẫn phải được giao hoặc admin huỷ + hoàn; đơn chưa thanh toán hết hạn như bình thường; tiền đã bán vẫn được chi trả.
BR-26	**Múi giờ:** mọi thời gian lưu và so sánh theo giờ Việt Nam (`Asia/Ho_Chi_Minh`, +07:00) ở cả MySQL, JDBC, JVM và JSON trả về.

5. Trạng thái, quy trình và dòng tiền

5.1 Trạng thái đơn hàng

PENDING_PAYMENT → VNPay thành công → CONFIRMED → nhập mã giao hàng → COMPLETED

PENDING_PAYMENT → quá 15 phút chưa trả / huỷ thanh toán → EXPIRED

| CONFIRMED → cửa hàng / admin huỷ → CANCELLED | CONFIRMED → quá giờ → NO_SHOW

Trạng thái thanh toán đi kèm: `UNPAID → PAID`; khi huỷ đơn đã trả tiền: `PAID → REFUND_PENDING → REFUNDED` (admin xác nhận).

5.2 Trạng thái túi và cửa hàng

AVAILABLE ⇄ hết / có đơn trả lại ⇄ SOLD_OUT → quá giờ → EXPIRED | chủ huỷ → CANCELLED

PENDING → admin duyệt → APPROVED ⇄ LOCKED | PENDING → REJECTED

5.3 Quy trình giỏ hàng, đặt hàng và thanh toán

1. Khách thêm các túi của một cửa hàng vào giỏ, mở giỏ, bấm "Đặt hàng". React gọi `POST /api/orders` (backend lấy dữ liệu từ giỏ của khách, không tin giá gửi lên).
2. Backend, trong **1 transaction**: kiểm tra từng túi còn bán và khung giờ còn giao nhau; trừ kho từng túi bằng UPDATE có điều kiện (sắp theo id túi để tránh deadlock) và ghi sổ kho `ORDERED`; thiếu 1 túi thì rollback toàn bộ; tạo `orders` + `order_items` (giá chụp lại), mã nhận hàng, `payments` PENDING; xoá giỏ; trả về link VNPay.
3. React chuyển trình duyệt sang VNPay. Khách nhập thẻ test và OTP.
4. VNPay chuyển về trang `/payment/result` của React kèm kết quả. React gửi toàn bộ tham số về backend; backend kiểm tra chữ ký, số tiền rồi cập nhật đơn sang `CONFIRMED`. Nếu có IPN (khi deploy hoặc dùng ngrok), IPN cũng gọi cùng một hàm xử lý, và hàm này không xử lý lại đơn đã xác nhận.
5. Trang kết quả hiện "Đặt thành công" và mã nhận hàng.

5.4 Quy trình nhận hàng

1. Khách đến quầy trong khung giờ, mở "Đơn của tôi" và đọc mã (hoặc đưa QR).
2. Nhân viên nhập mã → backend kiểm tra (BR-09) → hiện tên khách và danh sách túi trong đơn.
3. Nhân viên đưa túi, bấm "Đã giao" → đơn `COMPLETED`, ghi người giao. Khách thấy nút "Đánh giá".

5.5 Hoàn tiền cho khách

- **Khi nào:** chủ cửa hàng huỷ đơn (hết hàng), chủ cửa hàng huỷ cả túi, hoặc admin huỷ đơn; chỉ huỷ được đơn `CONFIRMED` (đã trả tiền, chưa giao), khách không tự huỷ đơn đã trả tiền. Ngoài ra hệ thống tự tạo yêu cầu hoàn khi **tiền về muộn** mà túi đã hết (BR-19, lý do `LATE_PAYMENT`).
- **Hoàn bao nhiêu:** luôn hoàn **toàn bộ** `total_amount`. Mỗi đơn hoàn tối đa một lần.

Huỷ đơn (lý do) → Đơn CANCELLED + REFUND_PENDING, trả số lượng, tạo refunds PENDING → Admin hoàn trên VNPay Sandbox → Nhập mã giao dịch hoàn: refunds COMPLETED, đơn REFUNDED

1. Mọi chỗ huỷ đơn đều gọi chung một hàm `RefundService.request(order, reason)`. Trong 1 transaction, hàm này đổi trạng thái đơn, trả số lượng cho các túi trong đơn (ghi sổ kho `RETURNED`), tạo bản ghi `refunds` (mã `RF-<mã đơn>`, số tiền, lý do, người huỷ) và ghi nhật ký.
2. Admin mở trang **Hoàn tiền**, thấy danh sách chờ hoàn kèm mã giao dịch VNPay gốc (`payments.gateway_txn_no`), thực hiện hoàn trên trang quản lý merchant VNPay Sandbox.
3. Admin nhập **mã giao dịch hoàn** rồi bấm "Xác nhận đã hoàn". Hệ thống không cho xác nhận nếu thiếu mã này.

5.6 Chi trả cho cửa hàng theo kỳ

Admin chi trả theo **kỳ** (ví dụ mỗi thứ Hai chi cho tuần trước). Mỗi kỳ chi trả **gắn với danh sách đơn cụ thể** (cột `orders.payout_id`), nên luôn trả lời được "khoản tiền này là của những đơn nào" và một đơn không bao giờ bị chi hai lần.

Chọn cửa hàng + khoảng ngày → **Xem trước danh sách đơn và tổng tiền** →
Tạo kỳ PENDING (gắn đơn vào kỳ) → **Chuyển khoản ngân hàng** → **Nhập mã chuyển khoản: PAID**

1. **Đơn đủ điều kiện:** thuộc cửa hàng đó, trạng thái `COMPLETED` hoặc `NO_SHOW`, `completed_at` nằm trong kỳ, **chưa thuộc kỳ nào** (`payout_id IS NULL`).
2. **Tạo kỳ:** hệ thống tính `order_count`, `gross_amount` (tổng khách trả), `commission_amount` (tổng phí), `net_amount = gross − commission`; chụp lại tài khoản ngân hàng hiện tại của cửa hàng; gắn `payout_id` cho các đơn bằng `UPDATE ... WHERE payout_id IS NULL`. Hai admin bấm cùng lúc thì đơn vẫn chỉ vào một kỳ.
3. **Xác nhận đã chi:** admin chuyển đúng `net_amount` vào tài khoản đã chụp, nhập mã giao dịch ngân hàng → `PAID`. Kỳ đã `PAID` không sửa và không huỷ được.
4. **Nhập sai:** kỳ còn `PENDING` thì huỷ được (`CANCELLED`), các đơn được gỡ khỏi kỳ để gom lại sau.

Cửa hàng yêu cầu rút tiền: chủ cửa hàng bấm "Yêu cầu rút tiền" → hệ thống dùng cùng hàm tạo kỳ ở bước 2 với khoảng ngày từ đơn chưa chi cũ nhất đến hôm nay, `created_by` là chủ cửa hàng → kỳ `PENDING` hiện trong danh sách của admin → admin chuyển khoản và nhập mã như bước 3. Không cần bảng riêng cho yêu cầu rút tiền.

5.7 Công thức đối soát

Mọi đồng tiền khách đã trả phải nằm ở đúng một trong các khoản sau. Trang **Đối soát** của admin tính từng khoản từ CSDL và hiện dòng kiểm tra; **chênh lệch phải bằng 0**, khác 0 là có lỗi dữ liệu.

#	Khoản	Cách tính	Dữ liệu mẫu
A	**Tiền thu từ khách**	Tổng `payments.amount` có `status = SUCCESS`	**464.000**
1	Đã hoàn cho khách	Tổng `refunds.amount`, `status = COMPLETED`	0
2	Đang chờ hoàn	Tổng `refunds.amount`, `status = PENDING`	45.000
3	Đã chi cho cửa hàng	Tổng `payouts.net_amount`, `status = PAID`	170.000
4	Đang chờ chi (kỳ PENDING)	Tổng `payouts.net_amount`, `status = PENDING`	0
5	Còn nợ cửa hàng (chưa vào kỳ)	Tổng `store_earning` đơn COMPLETED/NO_SHOW có `payout_id IS NULL`	18.700
6	Phí nền tảng đã hưởng	Tổng `commission_amount` đơn COMPLETED/NO_SHOW	33.300
7	Tiền đơn chưa giao	Tổng `total_amount` đơn `CONFIRMED` (chưa biết sẽ giao hay huỷ)	197.000
	Kiểm tra	`A − (1 + 2 + 3 + 4 + 5 + 6 + 7)`	**0**

Số dư tài khoản nền tảng lúc này = A − 1 − 3 (tiền đã thu trừ tiền đã trả ra). Trong đó, phần thực sự là doanh thu của nền tảng chỉ là khoản 6; các khoản 2, 4, 5, 7 là tiền đang giữ hộ khách hoặc cửa hàng.

5.8 Sao kê cho cửa hàng

Trang Doanh thu của chủ cửa hàng có **sao kê từng đơn**, mỗi dòng: mã đơn, ngày hoàn tất, khách trả, tỉ lệ phí, phí, thực nhận, trạng thái (đã chi ở kỳ `PO-...` hoặc chưa chi). Bên dưới là danh sách các kỳ chi trả (số đơn, số tiền, tài khoản nhận, mã chuyển khoản, ngày chi). Chủ cửa hàng tự đối chiếu được với sao kê ngân hàng của mình.

5.9 Nhật ký thao tác

Bảng `audit_logs` ghi lại mọi thao tác liên quan đến tiền và quản trị: **ai** (hoặc hệ thống), **làm gì**, **trên đối tượng nào**, **số tiền**, **lúc nào**. Chỉ thêm, không sửa, không xoá: CSDL có trigger chặn mọi lệnh UPDATE / DELETE trên bảng này (sổ kho `stock_movements` cũng vậy).

Nhóm	Hành động được ghi
Thanh toán, đơn hàng	`PAYMENT_SUCCESS` (hệ thống), `ORDER_CANCELLED`, `ORDER_EXPIRED`, `ORDER_COMPLETED`, `ORDER_NO_SHOW`
Hoàn tiền	`REFUND_REQUESTED`, `REFUND_COMPLETED`
Chi trả	`PAYOUT_CREATED`, `PAYOUT_PAID`, `PAYOUT_CANCELLED`
Quản trị	`BATCH_CANCELLED`, `STORE_APPROVED`, `STORE_REJECTED`, `STORE_LOCKED`, `STORE_RATE_CHANGED`, `USER_LOCKED`, `USER_UNLOCKED`, `USER_PASSWORD_RESET`

Backend có một hàm dùng chung `AuditService.log(action, entityType, entityId, amount, detail)`, tự lấy người thực hiện từ JWT; được gọi trong cùng transaction với thao tác chính.

5.10 Đối chiếu nghiệp vụ và CSDL

Bảng dưới kiểm tra từng nghiệp vụ cơ bản đã có đủ chỗ lưu trong CSDL và có ràng buộc hoặc câu kiểm tra đi kèm. File `foodrescue_checks.sql` chứa 19 câu truy vấn (C0-C13, thêm C1b, C1c, C3b, C3c, C3d): chạy sau khi test hoặc trước khi demo; dữ liệu đúng thì các câu C1-C12 trả về 0 dòng và C0 có chênh lệch bằng 0. Đã chạy trên **MySQL 8.0.40** (chế độ strict mặc định) và MariaDB 10.4 của XAMPP: dữ liệu mẫu sạch; đã thử cố ý làm sai tồn kho, tiền giao dịch, tổng đơn, giỏ hàng: các câu tương ứng đều báo lỗi.

Nghiệp vụ	Lưu ở đâu	Ràng buộc trong CSDL / câu kiểm tra
Đăng bán và tồn kho		
Đăng bán túi	`rescue_batches`: giá gốc, giá bán, số lượng, khung giờ, danh mục, ảnh	`chk_batch_price` (giảm ≥ 40%), `chk_batch_time`, `chk_batch_qty`
Thêm / bớt túi	`quantity` và `remaining_quantity` cùng cộng/trừ (BR-18)	`chk_batch_qty`; C1: `quantity − remaining` = số túi đã bán qua app; C2: trạng thái SOLD_OUT đúng với số còn lại
Sổ kho	`stock_movements` (LISTED, ADDED, REMOVED, ORDERED, RETURNED)	Trigger chặn sửa / xoá; `chk_stock_sign` (dấu đúng với lý do), `chk_stock_order`; C1b: tổng biến động = số còn lại; C1c: mỗi túi trong đơn có đúng dòng trừ / trả kho
Không bán vượt	UPDATE có điều kiện `remaining_quantity >= :qty`	`chk_batch_qty` (không âm); test 10 luồng (TC01)
Đặt hàng và thanh toán		
Giỏ hàng	`cart_items` (khoá chính khách + túi)	`chk_cart_qty` (1-5); C3d: giỏ chỉ có túi của 1 cửa hàng
Đơn nhiều túi	`order_items` (giá chụp lại lúc mua)	`chk_items_money`, `uq_items_batch`; C3b: tổng đơn = tổng chi tiết; C3c: khung giờ đơn nằm trong khung giờ mọi túi
Tiền khách trả	`order_items.unit_price` (giá lúc mua), `orders.total_amount`; `payments` (giao dịch VNPay)	`chk_order_money`; C4: đơn đã trả có đúng 1 giao dịch SUCCESS bằng đúng số tiền

Nghiệp vụ	Lưu ở đâu	Ràng buộc trong CSDL / câu kiểm tra
Trạng thái đơn và thanh toán	`orders.status` + `payment_status`	C5: hai trạng thái phải khớp nhau; `chk_order_done_time` : COMPLETED/NO_SHOW phải có thời điểm chốt
Đơn đúng cửa hàng	`orders.store_id` , `order_items.batch_id`	C3: mọi túi trong đơn thuộc cửa hàng của đơn
Giao hàng	`orders.handled_by` , `completed_at`	C11: người giao là chủ hoặc nhân viên của đúng cửa hàng
Tiền của nền tảng và cửa hàng		
% phí của admin	`stores.commission_rate` (admin đặt) → copy vào `orders.commission_rate` ; `commission_amount`	`chk_store_rate` (0-50%); `chk_order_money` : phí = round(total × rate / 100)
Tiền cửa hàng nhận	`orders.store_earning`	`chk_order_money` : thực nhận = total − phí
Rút tiền / chi trả	`payouts` (admin tạo theo kỳ hoặc cửa hàng yêu cầu) + `orders.payout_id`	`chk_payout_money` , `chk_payout_paid` , `chk_order_payout` ; C7: số tiền kỳ = tổng các đơn trong kỳ; C8, C9
Hoàn tiền	`refunds` (1 đơn tối đa 1 lần)	`refunds.order_id` UNIQUE, `chk_refund_done` ; C6: đơn chờ hoàn/đã hoàn khớp bản ghi hoàn tiền
Đối soát	Tính từ `payments` , `refunds` , `payouts` , `orders`	C0: chênh lệch = 0
Nhật ký	`audit_logs`	Trigger chặn sửa / xoá; C12: thanh toán, hoàn tiền, chi trả đều có dòng nhật ký
Mật khẩu tạm	`users.must_change_password`	Kiểm tra ở backend khi đăng nhập / gọi API (BR-01)
Thời gian	Mọi cột `DATETIME` theo giờ Việt Nam	Cấu hình múi giờ mục 7.6 (BR-26)
Tài khoản		
Chủ và nhân viên	`stores.owner_id` UNIQUE; `users.store_id`	`chk_users_staff` : nhân viên bắt buộc có cửa hàng; một chủ chỉ một cửa hàng
Đánh giá	`reviews` (1 đơn 1 đánh giá)	C10: chỉ đơn COMPLETED, đúng khách, đúng cửa hàng

6. Tích hợp API bên ngoài

Nhu cầu	Dịch vụ	Ghi chú
Thanh toán	**VNPay Sandbox**	Miễn phí, đăng ký tài khoản merchant test. Gọi từ backend.
Tính khoảng cách, thời gian	**OSRM Table API**	Không cần key. Gọi từ backend, có dự phòng khi lỗi.
Hiển thị bản đồ	**Leaflet + OpenStreetMap**	Không cần key. Dùng thư viện react-leaflet.
Vị trí người dùng	Geolocation API của trình duyệt	Chỉ chạy trên HTTPS hoặc localhost. Người dùng từ chối thì bấm chọn trên bản đồ.
Chỉ đường	Link Google Maps	`https://www.google.com/maps/dir/?api=1&destination={lat},{lng}`, không cần key.

6.1 VNPay Sandbox

a) Chuẩn bị

- Đăng ký merchant test trên trang sandbox của VNPay, nhận `vnp_TmnCode` và `vnp_HashSecret` qua email.
- URL thanh toán: `https://sandbox.vnpayment.vn/paymentv2/vpcpay.html`
- Key đặt trong biến môi trường, không đưa lên GitHub.

b) Tạo link thanh toán

Tham số	Giá trị
`vnp_Version`, `vnp_Command`	`2.1.0`, `pay`
`vnp_TmnCode`	Mã website sandbox
`vnp_Amount`	Số tiền × 100 (45.000đ → `4500000`)
`vnp_CurrCode`, `vnp_Locale`	`VND`, `vn`
`vnp_TxnRef`	Mã giao dịch duy nhất (`payments.txn_ref`)
`vnp_OrderInfo`, `vnp_OrderType`	"Thanh toan don FR260925-0004", `other`
`vnp_ReturnUrl`	`http://localhost:5173/payment/result` (trang React)
`vnp_IpAddr`	IP của khách
`vnp_CreateDate`, `vnp_ExpireDate`	`yyyyMMddHHmmss` giờ Việt Nam; hết hạn sau 10 phút
`vnp_SecureHash`	HMAC-SHA512 (khoá `vnp_HashSecret`) của chuỗi tham số đã **sắp xếp theo tên**, URL-encode, nối bằng `&`

c) Xác nhận kết quả

```
// PaymentService.confirm(params) - dùng chung cho Return URL và IPN
@Transactional
public Order confirm(Map<String, String> params) {
    if (!vnPaySigner.verify(params)) throw new BusinessException("Sai chữ ký VNPay");
    Payment p = paymentRepo.findByTxnRef(params.get("vnp_TxnRef")).orElseThrow();
    if (p.getAmount().longValue() * 100 != Long.parseLong(params.get("vnp_Amount")))
        throw new BusinessException("Sai số tiền");
    if (p.getStatus() != PaymentStatus.PENDING) return p.getOrder();     // đã xử lý rồi -> bỏ qua
    if ("00".equals(params.get("vnp_ResponseCode"))) {
        p.markSuccess(params);                 // SUCCESS, paid_at, gateway_txn_no, bank_code
        p.getOrder().markPaid();               // CONFIRMED + PAID
    } else {
        p.markFailed(params.get("vnp_ResponseCode"));
        orderService.expireAndReturnStock(p.getOrder());   // EXPIRED, cộng lại số lượng
    }
    return p.getOrder();
}
```

Mã `vnp_ResponseCode`: `00` thành công, `24` khách huỷ giao dịch, các mã khác là thất bại. Khi chạy trên máy cá nhân, VNPay không gọi được IPN về `localhost`, nên hệ thống xác nhận qua Return URL (vẫn kiểm tra chữ ký). Khi deploy thì khai báo thêm IPN URL `/api/payments/vnpay/ipn`.

d) Thẻ test

Ngân hàng **NCB** · Số thẻ `9704198526191432198` · Tên **NGUYEN VAN A** · Ngày phát hành **07/15** · OTP `123456`.
(Theo tài liệu sandbox công khai của VNPay; kiểm tra lại khi tích hợp.)

6.2 Bản đồ và tính khoảng cách

a) Cách tính

Bước	Cách làm
1. Lọc	MySQL lọc cửa hàng đã duyệt, có túi đang bán, trong bán kính R (đường chim bay, hàm `ST_Distance_Sphere`); lấy tối đa 30 cửa hàng gần nhất.
2. Khoảng cách thật	Gọi **1 lần** OSRM Table API cho cả 30 cửa hàng để lấy quãng đường (mét) và thời gian (giây) theo đường đi thực tế; sắp xếp lại.
3. Dự phòng	OSRM lỗi hoặc quá 2 giây: dùng khoảng cách chim bay × 1,3, đánh dấu `estimated = true`, React hiện "~2,4 km". Website vẫn chạy bình thường.

```
-- Bước 1 (kết quả tính bằng mét)
SELECT s.*, ST_Distance_Sphere(POINT(s.longitude, s.latitude), POINT(:lng, :lat)) AS air_m
FROM stores s
WHERE s.status = 'APPROVED'
  AND EXISTS (SELECT 1 FROM rescue_batches b WHERE b.store_id = s.id
              AND b.status = 'AVAILABLE' AND b.pickup_end > NOW())
HAVING air_m <= :radiusM
ORDER BY air_m LIMIT 30;

-- Bước 2: OSRM (thứ tự toạ độ là KINH ĐỘ,VĨ ĐỘ; điểm 0 là vị trí khách)
GET http://router.project-osrm.org/table/v1/driving/106.69,10.77;106.671239,10.758372;106.698942,10.781254
    ?sources=0&annotations=distance,duration
→ { "distances": [[0, 2350.4, 1810.2]], "durations": [[0, 452.1, 380.7]] }
```

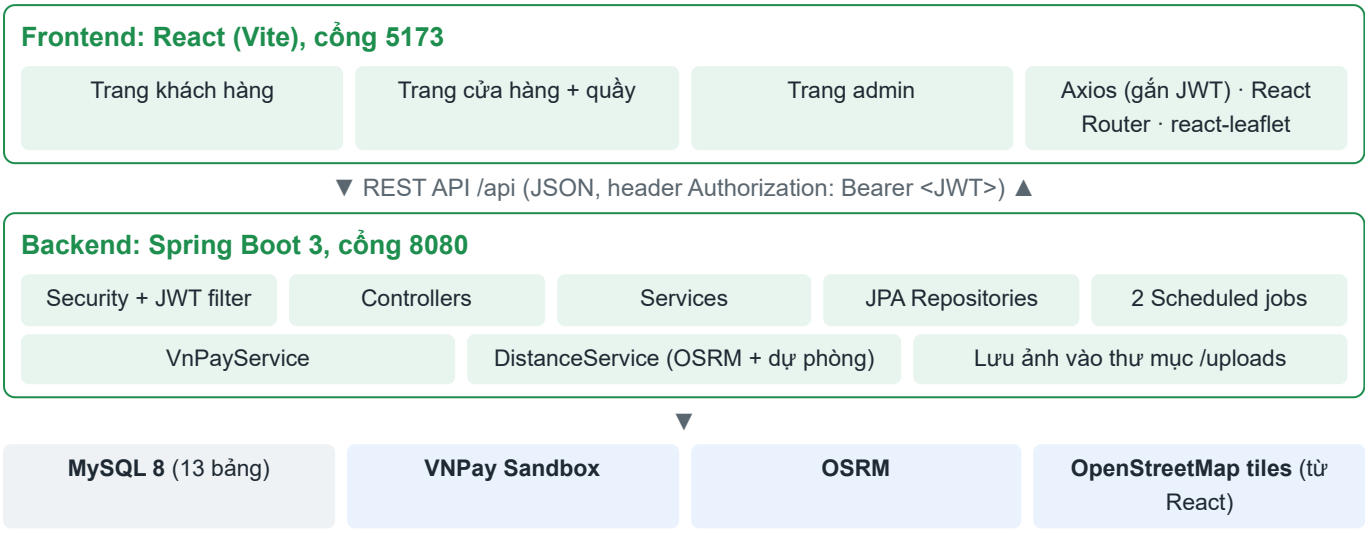
Server OSRM công khai chỉ dành cho mức dùng nhẹ và chỉ có profile ô tô; đủ cho đồ án. Muốn tính cho xe máy thì có thể thay bằng Goong Distance Matrix (cần key).

b) Phía React

- `navigator.geolocation.getCurrentPosition` lấy vị trí khách; lỗi thì dùng trung tâm TP.HCM và cho bấm chọn trên bản đồ.
- `react-leaflet`: tile `https://tile.openstreetmap.org/{z}/{x}/{y}.png`, ghi "© OpenStreetMap contributors"; marker cửa hàng, popup xem nhanh túi.
- Form đăng ký cửa hàng: bản đồ cho chủ quán bấm để ghim vị trí, lấy `latitude` / `longitude`.

7. Kiến trúc và công nghệ

7.1 Kiến trúc



7.2 Công nghệ

Tầng	Thư viện
Backend	Java 17, Spring Boot 3 (Web, Data JPA, Security, Validation), jjwt, Lombok, MySQL Connector/J, Maven. Tuỳ chọn: springdoc-openapi (Swagger).
Frontend	React 18 + Vite, React Router, Axios, **Ant Design** (giao diện, bảng, form), react-leaflet, qrcode.react, Recharts (biểu đồ doanh thu), dayjs.
CSDL	MySQL 8.0.16+ hoặc MariaDB 10.4+ của XAMPP (quản lý bằng phpMyAdmin hoặc MySQL Workbench). CI kiểm tra trên MySQL 8.
Công cụ	GitHub, Postman, Trello, Figma (tuỳ chọn).

7.3 Đăng nhập với JWT (đơn giản)

- `POST /api/auth/login` trả về `{ token, mustChangePassword, user }` (Quy ước chung 5.5). Token hạn 24 giờ và **chỉ chứa id người dùng**. React lưu token trong `localStorage`; `axios.js` tự gắn header `Authorization: Bearer <token>`.
- Backend: `JwtAuthFilter` đọc token rồi **đọc lại người dùng từ CSDL ở mỗi request** (vai trò, `storeId`, trạng thái, mật khẩu tạm), nên khoá tài khoản hoặc đặt lại mật khẩu có hiệu lực ngay, không phải đợi token hết hạn. Tài khoản bị khoá nhận 403 `ACCOUNT_LOCKED`; tài khoản đang dùng mật khẩu tạm chỉ gọi được `GET /api/me` và `PUT /api/me/password`, còn lại 403 `PASSWORD_CHANGE_REQUIRED` (BR-01).
- Quyền theo khu vực URL nằm trong `SecurityConfig` (mục 8); quyền chi tiết hơn dùng `@PreAuthorize`. Service lấy người đang đăng nhập bằng `CurrentUser.id()` và `CurrentUser.storeId()`, không tin `storeId` gửi lên.
- **DevAuth** (chỉ máy dev): khi `app.dev-auth.enabled: true` trong `application-local.yml`, gửi header `X-Dev-User-Id: 8` là gọi API với tư cách người dùng id 8 mà không cần đăng nhập (1 admin, 2 chủ An Phát, 6 nhân viên An Phát, 8 khách). Mặc định tắt trong `application.yml`.
- Gặp lỗi 401 thì React xoá token và chuyển về trang đăng nhập. CORS cho phép `http://localhost:5173`.

7.4 Chống bán vượt số lượng

Hai người cùng đặt túi cuối cùng: nếu đọc số còn lại rồi mới trừ trong Java thì cả hai đều thấy còn 1. Cách xử lý là trừ bằng một câu UPDATE có điều kiện, MySQL đảm bảo chỉ một người thành công:

```
@Modifying
@Query("""
    UPDATE RescueBatch b
       SET b.remainingQuantity = b.remainingQuantity - :qty,
           b.status = CASE WHEN b.remainingQuantity - :qty = 0 THEN 'SOLD_OUT' ELSE b.status END
     WHERE b.id = :id AND b.status = 'AVAILABLE' AND b.remainingQuantity >= :qty
""")
int decrementStock(Long id, int qty);  // trả về 0 → hết hàng → báo lỗi "Túi vừa được giải cứu hết"
```

CSDL có thêm ràng buộc `CHECK (remaining_quantity >= 0)` để chặn lần cuối.

Đơn nhiều túi: gọi `decrementStock` lần lượt cho từng túi **theo thứ tự id tăng dần** (hai đơn cùng lúc khoá túi theo cùng thứ tự nên không deadlock) trong cùng một `@Transactional`. Một túi trả về 0 thì ném lỗi, transaction rollback, các túi đã trừ trước đó được trả lại tự động. Mỗi lần trừ ghi 1 dòng sổ kho `ORDERED` trong cùng transaction.

7.5 Cấu trúc thư mục và chức năng từng file

Backend tổ chức theo **3 lớp**, mỗi lớp là một package, không chia theo module hay theo người. Nhìn tên thư mục là biết file thuộc lớp nào. Frontend React đóng vai trò View: chỉ gọi REST API và hiển thị JSON.

React (View)
pages/ · components/ · services/api.js

▼ HTTP JSON (Authorization: Bearer <JWT>)

Lớp 1: Controller `controller/`
Nhận request, kiểm tra `@Valid`, lấy người dùng bằng `CurrentUser`, gọi Service, trả DTO

▼

Lớp 2: Service `service/`
Toàn bộ nghiệp vụ và quy tắc BR-xx, `@Transactional`, ném `BusinessException`, chuyển Entity → DTO

▼

Lớp 3: Repository `repository/`
Spring Data JPA: chỉ truy vấn / lưu dữ liệu, không chứa nghiệp vụ

▼

MySQL 8
13 bảng, CHECK, trigger (Phụ lục A)

Quy tắc giữa các lớp	Giải thích
Controller chỉ gọi Service	Controller không gọi thẳng Repository, không viết `if` nghiệp vụ, không mở transaction.
Service không trả Entity	Trả record trong `dto/response/`; tránh vòng lặp JSON, lộ `password_hash`, lỗi lazy loading.
Service dùng chung	Đổi số lượng túi chỉ qua `StockService` (BR-23); ghi nhật ký qua `AuditService` (BR-17). Service khác gọi các hàm này, không tự viết lại.
Tác vụ tự động	`scheduler/` là điểm vào giống Controller: chỉ chọn việc cần làm theo thời gian rồi gọi Service.
Đồ dùng chung	`entity/`, `repository/`, `config/`, `security/`, `exception/`, `util/` dùng cho mọi chức năng; sửa file có sẵn thì mở PR nhãn `shared-change`.

✓ đã có trong repo (ngày 17/10/2026) · ○ cần tạo, kèm tuần dự kiến theo kế hoạch mục 11 (T1-T5). Tên file ○ là tên cả nhóm thống nhất; cần thêm DTO hoặc repository thì đặt vào đúng package của lớp đó.

a) Thư mục gốc

Đường dẫn	Chức năng	Trạng thái
foodrescue/	1 repo GitHub cho cả nhóm. Nhánh `main` (bản demo), `develop` (tích hợp), `feature/<mã việc>-<mô tả>`	✓
├── .github/workflows/ci.yml	CI tự chạy khi push hoặc mở PR: dựng MySQL 8, nạp schema, kiểm tra đối soát C0 = 0, `mvn verify`; frontend `npm ci`, lint, build	✓
├── .github/pull_request_template.md	Checklist khi mở PR: đặt tên, API, giao diện, file dùng chung, test	✓
├── .editorconfig · .gitattributes	UTF-8, xuống dòng LF, thụt lề 2 (Java 4) cho mọi máy	✓
├── .gitignore	Không đưa lên Git: `target/`, `node_modules/`, `dist/`, `uploads/`, `application-local.yml`, `.env`	✓
├── README.md	Cài đặt, chạy dự án, cấu trúc 3 lớp, quy trình Git, lỗi hay gặp	✓
├── database/foodrescue_schema.sql	Nguồn duy nhất của CSDL: 13 bảng, 25 CHECK, 4 trigger, dữ liệu mẫu (mật khẩu 123456). Chạy lại là xoá và tạo lại `foodrescuedb`	✓
├── database/foodrescue_checks.sql	19 câu kiểm tra dữ liệu (C0-C13, C1b, C1c, C3b, C3c, C3d); đúng thì C0 = 0 và C1-C12 không trả dòng nào	✓
├── database/demo_data.sql	Dữ liệu demo: khoảng 15 cửa hàng, 30 túi tối nay, kèm sổ kho `LISTED`	○ T3
├── docs/	Tài liệu PDF, checklist công việc, ảnh chụp màn hình cho báo cáo	✓
│ └── postman/	Collection Postman chung, biến `{{baseUrl}}`, `{{token}}`, mỗi nhóm API một thư mục	○ T1
├── backend/	Spring Boot 3.3.5, Java 17, Maven (mục b)	✓
└── frontend/	React 18 + Vite + Ant Design (mục c)	✓

b) Backend: `backend/src/main/java/com/foodrescue/`

Đường dẫn	Chức năng	Trạng thái
backend/pom.xml	Thư viện: Web, Data JPA, Security, Validation, MySQL, jjwt 0.12, springdoc (Swagger), Lombok. Thêm thư viện bằng PR riêng	✓
BackendApplication.java	Điểm khởi động; `@EnableScheduling` bật tác vụ tự động	✓
`config/` : cấu hình Spring		
├── SecurityConfig.java	Quyền theo khu vực URL (mục 8), CORS cho `localhost:5173`, trả 401 / 403 dạng ApiError, gắn `JwtAuthFilter`, bật `@PreAuthorize`, mã hoá BCrypt	✓
├── TimeZoneConfig.java	Đặt JVM theo giờ Việt Nam (BR-26)	✓
└── WebConfig.java	Phục vụ ảnh đã tải lên tại `/uploads/**`	○ T1
`controller/` : LỚP 1, REST API công khai và phía khách hàng		
├── PingController.java	`GET /api/ping`: kiểm tra backend đang chạy	✓

Đường dẫn	Chức năng	Trạng thái
├─ MeController.java	`GET /api/me` (đã có); thêm `PUT /api/me` (sửa hồ sơ), `PUT /api/me/password` (đổi mật khẩu)	✓ / ○ T1
├─ AuthController.java	`POST /api/auth/register` (khách), `POST /api/auth/login` (trả token + user)	○ T1
├─ StoreRegistrationController.java	`POST /api/auth/register-store` (multipart): tạo chủ + cửa hàng `PENDING`	○ T2
├─ CategoryController.java	`GET /api/categories` theo `sort_order`	○ T2
├─ BatchController.java	`GET /api/batches/{id}` chi tiết túi + đánh giá cửa hàng (đã có); thêm `GET /api/batches` (lọc, sắp xếp, khoảng cách)	✓ / ○ T3
├─ PublicStoreController.java	`GET /api/stores/{id}`, `/api/stores/{id}/reviews` (cửa hàng bị khoá trả 404)	○ T3
├─ CartController.java	`/api/cart`: xem, thêm, sửa số lượng, xoá 1 túi, xoá cả giỏ	✓
├─ OrderController.java	`POST /api/orders` (từ giỏ hoặc danh sách túi), `GET /api/orders`, `/{id}`, `POST /{id}/cancel` (đơn chưa trả)	✓
├─ PaymentController.java	`POST /api/payments/create-url`; `GET /api/payments/vnpay/return` và `/ipn` cùng gọi một hàm xác nhận; `POST /mock-success` chỉ để thử trên máy dev	✓
└─ ReviewController.java	`POST /api/orders/{id}/review`: chỉ đơn `COMPLETED` của mình, 1 lần	○ T4
`controller/store/`: khu vực cửa hàng `/api/store/**` (chủ và nhân viên)		
├─ StoreOrderController.java	`GET /api/store/orders`, `/{id}`; `POST /{id}/cancel` (đơn đã trả thì tạo yêu cầu hoàn tiền)	✓
├─ StorePickupController.java	`GET /api/store/pickup/lookup?code=`, `POST /api/store/pickup/handover`: đúng cửa hàng, đúng giờ (BR-09)	✓
├─ StoreBatchController.java	`/api/store/batches`: danh sách, tạo, sửa, huỷ túi (chỉ chủ); `PATCH .../quantity`, `GET .../stock-movements`	○ T2
├─ StoreProfileController.java	`GET/PUT /api/store/profile`: hồ sơ, vị trí, giờ mở cửa, tài khoản ngân hàng (chỉ chủ)	○ T2
├─ StoreStaffController.java	`/api/store/staff`: tạo nhân viên (mật khẩu tạm), khoá / mở khoá (chỉ chủ)	○ T4
└─ StoreRevenueController.java	`/api/store/revenue`, `/statement`, `/payouts`, `/payouts/request` (chỉ chủ)	○ T4
`controller/admin/`: khu vực quản trị `/api/admin/**`		
├─ AdminStoreController.java	`/api/admin/stores`: duyệt (kèm % phí), từ chối, khoá, mở khoá, đổi % phí	○ T2
├─ AdminCategoryController.java	CRUD `/api/admin/categories` (ảnh, thứ tự; không xoá danh mục đang có túi)	○ T2
├─ AdminUserController.java	`/api/admin/users`: lọc, khoá / mở khoá, đặt lại mật khẩu tạm	○ T4
├─ AdminOrderController.java	`/api/admin/orders`: lọc, tra mã, chi tiết kèm lịch sử tiền, huỷ đơn + hoàn tiền	○ T4

Đường dẫn	Chức năng	Trạng thái
├─ AdminRefundController.java	`/api/admin/refunds`: danh sách chờ hoàn, xác nhận đã hoàn (bắt buộc `gatewayRef`)	○ T4
├─ AdminPayoutController.java	`/api/admin/payouts`: tiền chưa chi, xem trước, tạo kỳ, xác nhận đã chi, huỷ kỳ	○ T4
├─ AdminFinanceController.java	`GET /api/admin/finance/reconciliation`: 7 khoản tiền, chênh lệch, bảng theo cửa hàng (mục 5.7)	○ T4
├─ AdminAuditLogController.java	`GET /api/admin/audit-logs`: lọc theo hành động, đối tượng, người, ngày	○ T4
└─ AdminDashboardController.java	`GET /api/admin/dashboard`: tổng đơn, GMV, túi đã cứu, số cửa hàng, số khách	○ T4
`service/`: LỚP 2, nghiệp vụ		
├─ AuditService.java	`log(actorId, action, entityType, entityId, amount, detail)`: ghi nhật ký trong cùng transaction	✓
├─ StockService.java	Ghi sổ kho `recordMovement(...)` (đã có); thêm hàm trừ / trả / điều chỉnh số lượng để mọi thay đổi số lượng đi qua đây (BR-23)	✓ / ○ T2
├─ CartService.java	Giỏ hàng: giá, số còn lại mới nhất, khung giờ chung; khác cửa hàng thì `CART_OTHER_STORE`	✓
├─ OrderService.java	Tạo đơn trong 1 transaction (khoá túi theo id tăng dần, tính phí qua `Money`, ghi sổ kho, xoá giỏ); đơn của khách / cửa hàng; huỷ đơn	✓
├─ PaymentService.java	Tạo link VNPay (hết hạn 10 phút), kiểm tra chữ ký + số tiền, xác nhận đơn chỉ 1 lần	✓
├─ PickupService.java	Tra mã nhận hàng, giao hàng (BR-09), ghi người giao	✓
├─ BatchService.java	Chi tiết túi (đã có); thêm tạo / sửa / huỷ túi, kiểm tra giá (`Money.isValidRescuePrice`), khung giờ, `BATCH_HAS_ORDERS`	✓ / ○ T2
├─ AuthService.java	Đăng ký (BCrypt, `EMAIL_EXISTS`), đăng nhập (`INVALID_CREDENTIALS`, `ACCOUNT_LOCKED`), hồ sơ, đổi mật khẩu (tắt mật khẩu tạm)	○ T1
├─ FileStorageService.java	`save(file)`: chỉ nhận jpg / png ≤ 5 MB, kiểm tra nội dung thật, lưu vào `uploads/`, trả URL	○ T1
├─ StoreService.java	Đăng ký, duyệt, khoá cửa hàng, hồ sơ; kiểm tra giờ, toạ độ, tài khoản ngân hàng; ghi log `STORE_*`	○ T2
├─ CategoryService.java	Danh mục cho khách và admin	○ T2
├─ CatalogService.java	Tìm túi theo vị trí, danh mục, bán kính, sắp xếp; trang cửa hàng công khai	○ T3
├─ DistanceService.java	Gọi OSRM 1 lần cho tối đa 30 cửa hàng, timeout 2 giây; lỗi thì chim bay × 1,3 và `estimated = true`	○ T3
├─ RefundService.java	`request(order, reason)`: mọi chỗ huỷ đơn đã trả tiền đều gọi (đổi trạng thái, trả kho, tạo refunds, ghi log); admin xác nhận đã hoàn	○ T4
├─ PayoutService.java	Tạo kỳ chi trả trong 1 transaction; `createForStore(storeId, from, to)` dùng chung cho admin và yêu cầu rút tiền	○ T4

Đường dẫn	Chức năng	Trạng thái
├── ReconciliationService.java	Tính 7 khoản tiền và chênh lệch đối soát (mục 5.7)	○ T4
├── StaffService.java · UserService.java	Tài khoản nhân viên (chủ quản lý); admin khoá / mở, đặt lại mật khẩu tạm (ghi log `USER_*`)	○ T4
├── RevenueService.java	Doanh thu, sao kê từng đơn, danh sách kỳ chi trả của cửa hàng	○ T4
└── ReviewService.java · DashboardService.java	Đánh giá đơn (BR-11); số liệu thống kê cho admin	○ T4
`repository/`: LỚP 3, truy cập dữ liệu		
├── (13 file) UserRepository, StoreRepository, RescueBatchRepository, OrderRepository ...	Spring Data JPA cho 13 entity (`findByEmail`, `findByOwnerId`, `findByTxnRef`, `findByIdForUpdate` khoá dòng túi ...). Cần truy vấn mới thì thêm hàm vào đúng repository	✓
└── BatchSearchRepository.java	Câu tìm túi theo vị trí (`ST_Distance_Sphere`), chỉ túi `AVAILABLE` của cửa hàng `APPROVED`	○ T3
`entity/` · `dto/`: dữ liệu (Model)		
entity/ (14 file)	Ánh xạ 13 bảng: `User`, `Store`, `Category`, `RescueBatch`, `CartItem` (+ `CartItemId` khoá ghép), `Order`, `OrderItem`, `StockMovement`, `Payment`, `Refund`, `Review`, `Payout`, `AuditLog`. Chạy `ddl-auto=validate` nên entity sai là không khởi động được	✓
entity/enums/ (13 file)	`Role`, `UserStatus`, `StoreStatus`, `BatchStatus`, `OrderStatus`, `PaymentStatus`, `PaymentTxnStatus`, `RefundStatus`, `RefundReason`, `PayoutStatus`, `StockReason`, `AuditAction`, `EntityType`. Giá trị giống hệt CSDL	✓
dto/request/	Dữ liệu client gửi lên, kiểm tra bằng `@Valid`. Đã có: `AddToCartRequest`, `UpdateCartItemRequest`, `CreateOrderRequest`, `CreateOrderItemRequest`, `CancelOrderRequest`, `CreatePaymentUrlRequest`, `HandoverRequest`. Thêm theo từng API mới (`LoginRequest`, `RegisterRequest`, `BatchRequest` ...)	✓ / ○
dto/response/	Dữ liệu trả về. Đã có: `ApiError` (định dạng lỗi duy nhất), `PageResponse` (phân trang), `CartResponse`, `CartItemDto`, `OrderResponse`, `OrderItemResponse`, `PaymentResponse`, `PaymentVerifyResponse`, `PickupLookupResponse`, `BatchDetailResponse`	✓ / ○
`security/` · `exception/` · `scheduler/` · `util/`		
security/JwtService.java	Tạo token (hạn 24 giờ, chỉ chứa id người dùng) và đọc id từ token	✓
security/JwtAuthFilter.java	Chạy ở mọi request: đọc `Authorization: Bearer` (hoặc `X-Dev-User-Id` khi bật DevAuth), chặn tài khoản bị khoá, chặn tài khoản đang dùng mật khẩu tạm	✓
security/AuthUserLoader.java · AuthUser.java	Đọc người dùng từ CSDL, tìm `storeId` đúng (chủ: qua `stores.owner_id`; nhân viên: `users.store_id`)	✓
security/CurrentUser.java	`CurrentUser.id()`, `CurrentUser.storeId()`, `CurrentUser.get()`. Luôn lấy `storeId` từ đây, không tin `storeId` gửi lên	✓

Đường dẫn	Chức năng	Trạng thái
`security/ApiErrorWriter.java`	Ghi lỗi ApiError từ filter (nơi GlobalExceptionHandler không bắt được)	✓
`exception/ErrorCode.java`	Mã lỗi, mỗi mã có HTTP status và câu thông báo tiếng Việt. Thiếu mã thì thêm qua PR nhỏ	✓
`exception/BusinessException.java`	Ném lỗi nghiệp vụ: `throw new BusinessException(ErrorCode.BATCH_SOLD_OUT)`	✓
`exception/GlobalExceptionHandler.java`	Chuyển mọi lỗi thành ApiError: nghiệp vụ, `@Valid` (kèm lỗi từng ô), vi phạm CSDL (422), URL không có (404), `@PreAuthorize` từ chối (403), lỗi khác (500)	✓
`scheduler/OrderScheduler.java`	Mỗi phút: đơn chưa trả quá hạn → `EXPIRED` + trả kho; túi quá giờ → `EXPIRED`; đơn quá giờ lấy 30 phút → `NO_SHOW`	✓
`util/Money.java`	Tính tiền duy nhất: phí, thực nhận, thành tiền, kiểm tra giá ≤ 60%, % giảm (khớp CHECK trong CSDL)	✓
`util/VNPayUtil.java`	Ký HMAC-SHA512 và dựng chuỗi tham số VNPay (sắp xếp, URL-encode)	✓
`resources/` và `test/`		
`resources/application.yml`	Cấu hình chung (commit): CSDL, múi giờ, `validate`, JWT, thời hạn đơn, VNPay, OSRM, `app.dev-auth.enabled`	✓
`resources/application-local.example.yml`	Mẫu cấu hình riêng (commit): mỗi người chép thành `application-local.yml`	✓
`resources/application-local.yml`	Không commit. Mật khẩu MySQL, JWT secret, key VNPay của máy mình	mỗi máy
`test/.../BackendApplicationTests.java`	Ứng dụng khởi động được, entity khớp CSDL (cần MySQL)	✓
`test/.../security/SecurityRulesTest.java`	9 trường hợp: 401, 403, DevAuth, Bearer token, tài khoản khoá, mật khẩu tạm (cần MySQL)	✓

`test/.../service/*ServiceTest.java`
`test/.../scheduler/OrderSchedulerTest.java`
`test/.../util/MoneyTest.java`
	Unit test Mockito: giỏ hàng, tạo đơn + chống bán vượt + huỷ đơn, VNPay, giao hàng, tác vụ tự động, công thức tiền (24 test)	✓

`test/.../service/(service mới)Test.java`	Mỗi service mới có unit test cho quy tắc chính; test 10 luồng giữ túi còn 1 → đúng 1 thành công (TC01)	○ T1-T5

c) Frontend: `frontend/`

Đường dẫn	Chức năng	Trạng thái
`package.json · package-lock.json`	React 18, antd, @ant-design/icons, axios, react-router-dom, dayjs, react-leaflet, qrcode.react, recharts. Luôn commit cả lock	✓
`vite.config.js · index.html`	Cổng 5173; chuyển `/api`, `/uploads` sang backend 8080; trang gốc	✓
`.prettierrc.json · .oxlintrc.json`	Format và kiểm tra code (CI chạy lint)	✓
`src/main.jsx`	Gắn theme Ant Design, tiếng Việt, `BrowserRouter`	✓
`src/App.jsx`	Hiện chuyển màn hình bằng state và chọn người dùng giả (DevAuth). T1 thay bằng `routes/AppRoutes.jsx` + trang đăng nhập	✓ / ○ T1

Đường dẫn	Chức năng	Trạng thái
src/routes/storeRoutes.jsx · storeMenu.js	Route và mục menu khu vực cửa hàng	✓
src/routes/AppRoutes.jsx · PrivateRoute.jsx	Khai báo mọi route; `<PrivateRoute roles={[...]}>`: chưa đăng nhập → `/login`, sai vai trò → về trang của vai trò	○ T1
src/layouts/	`CustomerLayout` (header, nội dung tối đa 1200px), `StoreLayout`, `AdminLayout` (menu trái, nút đăng xuất)	○ T1
`src/pages/`: màn hình (mỗi màn hình 1 file)		
pages/auth/	`LoginPage`, `RegisterPage`, `ChangePasswordPage`, `ProfilePage`	○ T1
pages/customer/	Đã có: `HomeCatalogView` (trang chủ, đang dùng dữ liệu giả), `RescueBatchDetailView`, `CartView`, `MyOrdersView`, `OrderInvoiceDetailView`, `PaymentResultView`. Thêm `MapPage`, `StorePage`	✓ / ○ T3
pages/store/	Đã có: `StoreOrdersView`, `StorePickupView` (màn hình quầy), `StoreBatchesPage`, `StoreRegisterPage` (2 trang sau đang dùng dữ liệu giả). Thêm `StoreProfilePage` (T2), `StoreStaffPage`, `StoreRevenuePage` (T4)	✓ / ○
pages/admin/	`AdminStoresPage`, `AdminCategoriesPage` (T2); `AdminUsersPage`, `AdminOrdersPage`, `AdminRefundsPage`, `AdminPayoutsPage`, `AdminFinancePage`, `AdminAuditLogsPage`, `AdminDashboardPage` (T4)	○ T2 / T4
Thành phần dùng chung		
src/components/store/	`BatchActions`, `BatchFormModal`, `LocationPicker` (ghim bản đồ), `RuleNote`, `StoreShell`	✓
src/components/	Component nhiều màn hình cùng dùng: `StatusTag`, `BatchCard`, `PriceTag`, `MoneyText`, `PickupWindow`, `EmptyState`, `ReviewButton` ...	○ T1-T4
src/services/axios.js	Mọi lời gọi API đi qua đây: gắn JWT, chuẩn hoá lỗi `{status, code, message, details}`, 401 → `/login`, cần đổi mật khẩu → `/change-password`; `toFormErrors(e)`	✓
src/services/api.js	Hàm gọi API giỏ hàng, đơn, thanh toán, quầy, đơn cửa hàng, chi tiết túi. API mới thêm file theo nhóm: `authApi.js`, `storeApi.js`, `catalogApi.js`, `adminApi.js`	✓ / ○
src/hooks/	`useAuth` (lưu đăng nhập trong localStorage), `useGeolocation`, `useCart`	○ T1-T3
src/constants/status.js	Nhãn tiếng Việt và màu của các nhóm trạng thái	✓
src/utils/format.js · geo.js	`formatMoney`, `discountPercent`, `formatDateTime`, `formatPickupWindow` ...; kiểm tra toạ độ trong Việt Nam	✓
src/utils/maps.js	`directionsUrl(lat, lng)`: link chỉ đường Google Maps	○ T3
src/theme/theme.js · stitchTheme.js	Màu, font, cỡ chữ, bo góc, theme Ant Design; theme riêng khu vực cửa hàng. Không viết mã màu trực tiếp trong component	✓
src/styles/store.css	CSS khu vực cửa hàng	✓
src/mocks/storeMockData.js	Dữ liệu giả tạm thời của trang túi giải cứu; xoá khi nối API thật	✓ (xoá T2)

d) Thêm file mới thì đặt ở đâu

Cần làm	Đặt ở
API mới	`controller/` (khu vực cửa hàng: `controller/store/`, admin: `controller/admin/`) + `service/XService` + record trong `dto/request`, `dto/response`. Đặt URL đúng tiền tố (mục 8) là có phân quyền; API chỉ dành cho chủ cửa hàng thêm `@PreAuthorize("hasRole('STORE_OWNER')")`
Câu truy vấn riêng	Thêm hàm vào repository của entity đó trong `repository/`, hoặc tạo repository mới cũng trong `repository/`
Đổi số lượng túi	Chỉ gọi `StockService`; không tự cộng / trừ `remainingQuantity`
Thêm bảng, cột, giá trị trạng thái	Sửa `foodrescue_schema.sql` + entity + enum trong cùng 1 PR `[DB]`
Mã lỗi mới	PR nhỏ thêm vào `exception/ErrorCode.java`
Trang mới	`pages/<customer|store|admin|auth>/XPage.jsx`, khai báo route trong `routes/`
Gọi API từ trang	Hàm trong `services/`, luôn dùng `api` từ `services/axios.js`
Component nhiều trang cùng dùng	`components/`, PR nhãn `shared-change`
Màu, trạng thái, cách hiện tiền / ngày	Chỉ dùng `theme/`, `constants/status.js`, `utils/format.js`; thiếu thì thêm vào đúng file đó

7.6 Chạy dự án trên máy

1. Chạy `database/foodrescue_schema.sql` trong MySQL (hoặc MySQL của XAMPP), rồi `foodrescue_checks.sql` để kiểm tra.
2. Backend: chép `application-local.example.yml` thành `application-local.yml` (cùng thư mục `resources/`, không commit), điền mật khẩu MySQL, JWT secret (≥ 32 ký tự), key VNPay → trong `backend/` chạy `mvn spring-boot:run` → mở `http://localhost:8080/swagger-ui.html`.
3. Frontend: trong `frontend/` chạy `npm install` → `npm run dev` → mở `http://localhost:5173`. Chi tiết từng bước và lỗi hay gặp: `README.md`.
4. Deploy để demo online (tuỳ chọn): frontend lên Vercel, backend lên Render/Railway, MySQL trên Railway/Aiven.

Cấu hình bắt buộc để thời gian và tiền đúng (BR-26):

```
# application.yml (đã có sẵn trong repo)
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/foodrescuedb?serverTimezone=Asia/Ho_Chi_Minh&characterEncoding=UTF-8
  jpa:
    hibernate.ddl-auto: validate            # bảng tạo từ foodrescue_schema.sql, không để Hibernate tự tạo
    properties.hibernate.jdbc.time_zone: Asia/Ho_Chi_Minh
  jackson:
    time-zone: Asia/Ho_Chi_Minh
    serialization.write-dates-as-timestamps: false   # trả "2026-09-25T21:00:00"

# Chạy JVM: java -Duser.timezone=Asia/Ho_Chi_Minh -jar app.jar
# (repo đã đặt sẵn trong config/TimeZoneConfig.java)
```

Script CSDL cần MySQL 8.0.16 trở lên (ràng buộc CHECK và trigger). Script đã chạy thử trên MySQL 8.0.40 (13 bảng, 25 ràng buộc CHECK, 4 trigger). XAMPP cài sẵn MariaDB 10.4: vẫn chạy được, nhưng nên dùng MySQL 8 cho giống môi trường nộp bài.

8. Danh sách API

Quyền: **P** công khai · **C** khách hàng · **S** nhân viên · **O** chủ cửa hàng · **A** admin.

> Khu vực URL đã chốt trong `SecurityConfig`: `/api/auth/**`, `/api/payments/vnpay/ipn` và `GET /api/batches|stores|categories/**` công khai · `/api/cart/**`, `/api/orders/**`, `/api/payments/**` khách hàng · `/api/store/**` chủ và nhân viên · `/api/admin/**` admin · còn lại (vd `/api/me`) chỉ cần đăng nhập. API ghi **O** trong khu `/api/store/**` phải thêm `@PreAuthorize("hasRole('STORE_OWNER')")` để nhân viên không gọi được (TC11).

Method	Endpoint	Mô tả	Quyền
Xác thực			
POST	`/api/auth/register`	Đăng ký khách hàng	P
POST	`/api/auth/register-store`	Đăng ký chủ cửa hàng + cửa hàng (multipart, có ảnh giấy phép)	P
POST	`/api/auth/login`	Đăng nhập, trả JWT	P
GET/PUT	`/api/me`, `/api/me/password`	Hồ sơ, đổi mật khẩu	Đăng nhập
Công khai			
GET	`/api/categories`	Danh mục	P
GET	`/api/batches?lat&lng&radiusKm&categoryId&sort`	Túi gần vị trí, kèm khoảng cách, thời gian	P
GET	`/api/batches/{id}?lat&lng`	Chi tiết túi	P
GET	`/api/stores/{id}`, `/api/stores/{id}/reviews`	Thông tin cửa hàng, đánh giá	P
Khách hàng			
GET	`/api/cart`	Giỏ hàng (giá, số còn lại mới nhất, đánh dấu túi hết)	C
POST/PUT/DELETE	`/api/cart/items` {batchId, quantity}, `/api/cart/items/{batchId}`, `/api/cart`	Thêm, sửa số lượng, xoá 1 túi, xoá hết giỏ	C
POST	`/api/orders`	Tạo đơn từ giỏ hàng, trả `paymentUrl`	C
GET	`/api/orders`, `/api/orders/{id}`	Đơn của tôi, chi tiết + mã nhận hàng	C
POST	`/api/orders/{id}/review`	Đánh giá	C
POST	`/api/orders/{id}/cancel`	Khách huỷ đơn **chưa thanh toán**	C
GET	`/api/payments/vnpay/return?{vnp_*}`	Xác nhận kết quả từ trang kết quả	C
GET	`/api/payments/vnpay/ipn`	VNPay gọi (khi deploy)	VNPay
Cửa hàng			
GET	`/api/store/orders?date&status`	Đơn của cửa hàng	S, O
GET	`/api/store/pickup/{code}`	Tra đơn theo mã nhận hàng	S, O
POST	`/api/store/pickup/{code}/complete`	Xác nhận đã giao	S, O
PATCH	`/api/store/batches/{id}/quantity` {delta}	Thêm (+) / bớt (−) túi	S, O
GET	`/api/store/batches/{id}/stock-movements`	Sổ kho của túi	S, O

Method	Endpoint	Mô tả	Quyền
GET/POST/PUT	`/api/store/batches`, `/api/store/batches/{id}`	Danh sách, tạo, sửa túi (multipart có ảnh)	O
POST	`/api/store/batches/{id}/cancel`	Huỷ túi (huỷ luôn các đơn đã trả tiền, chuyển chờ hoàn tiền)	O
POST	`/api/store/orders/{id}/cancel`	Huỷ đơn (lý do)	O
GET/PUT	`/api/store/profile`	Hồ sơ cửa hàng	O
GET/POST	`/api/store/staff`, `/api/store/staff/{id}/lock|unlock`	Nhân viên	O
GET	`/api/store/revenue?from&to`	Doanh thu tổng hợp, biểu đồ	O
GET	`/api/store/statement?from&to`	Sao kê từng đơn (thực nhận, kỳ chi trả)	O
GET	`/api/store/payouts`, `/api/store/payouts/{id}`	Các kỳ đã chi, chi tiết đơn trong kỳ	O
POST	`/api/store/payouts/request`	Yêu cầu rút tiền (tạo kỳ PENDING)	O
Admin			
GET	`/api/admin/dashboard?from&to`	Thống kê	A
GET/POST	`/api/admin/stores?status`, `/api/admin/stores/{id}/approve|reject|lock|unlock`	Duyệt (kèm tỉ lệ phí), khoá cửa hàng	A
PUT	`/api/admin/stores/{id}/commission` {rate}	Đổi tỉ lệ phí (áp dụng cho đơn mới)	A
GET/POST	`/api/admin/users`, `/api/admin/users/{id}/lock|unlock`	Người dùng	A
POST	`/api/admin/users/{id}/reset-password`	Đặt lại mật khẩu tạm (trả về cho admin 1 lần)	A
CRUD	`/api/admin/categories`	Danh mục	A
GET/POST	`/api/admin/orders?status&paymentStatus&code`, `/api/admin/orders/{id}`, `/api/admin/orders/{id}/cancel`	Đơn hàng + lịch sử tiền, huỷ đơn	A
GET/POST	`/api/admin/refunds?status`, `/api/admin/refunds/{id}/complete` {gatewayRef, note}	Danh sách hoàn tiền, xác nhận đã hoàn	A
GET	`/api/admin/payouts/unpaid-summary`	Tiền chưa chi của từng cửa hàng	A
GET	`/api/admin/payouts/preview?storeId&from&to`	Xem trước đơn đủ điều kiện và tổng tiền	A
GET/POST	`/api/admin/payouts` {storeId, from, to}, `/api/admin/payouts/{id}`	Tạo kỳ chi trả; danh sách, chi tiết kỳ	A
POST	`/api/admin/payouts/{id}/paid` {transferRef}, `/api/admin/payouts/{id}/cancel`	Xác nhận đã chuyển khoản; huỷ kỳ PENDING	A
GET	`/api/admin/finance/reconciliation?from&to`	Đối soát 7 khoản tiền + chênh lệch	A

Method	Endpoint	Mô tả	Quyền
GET	`/api/admin/audit-logs?action&entityType&entityId&from&to`	Nhật ký thao tác	A

Lỗi trả về thống nhất dạng `ApiError`: `{ "code": "BATCH_SOLD_OUT", "message": "Túi vừa được giải cứu hết", "details": [], "timestamp": "...", "path": "..." }` với HTTP 400, 401, 403, 404, 409, 422. Danh sách mã lỗi, quy ước đặt tên và giao diện: xem tài liệu **FoodRescue_QuyUocChung.pdf**.

9. Danh sách màn hình

Khách hàng

- `/` Trang chủ, danh sách túi (lọc, sắp xếp)
- `/map` Bản đồ quanh tôi
- `/batches/:id` Chi tiết túi, thêm vào giỏ
- `/cart` Giỏ hàng, đặt hàng
- `/payment/result` Kết quả thanh toán
- `/orders` Đơn của tôi (mã, QR, chỉ đường, đánh giá)
- `/stores/:id` Trang cửa hàng công khai
- `/login` · `/register` · `/profile` · `/change-password`

Nhân viên

- `/store/counter` Màn hình quầy: đơn hôm nay + ô nhập mã giao hàng
- `/store/batches` Thêm / bớt túi, xem sổ kho

Chủ cửa hàng

- `/store/register` Đăng ký cửa hàng (ghim bản đồ)
- `/store/counter` Màn hình quầy
- `/store/batches` Quản lý túi (form tạo/sửa)
- `/store/orders` Đơn hàng, huỷ đơn
- `/store/staff` Nhân viên
- `/store/revenue` Doanh thu, sao kê từng đơn, kỳ chi trả, đánh giá
- `/store/profile` Hồ sơ cửa hàng

Admin

- `/admin` Thống kê
- `/admin/stores` Duyệt / khoá cửa hàng
- `/admin/users` Người dùng
- `/admin/categories` Danh mục
- `/admin/orders` Đơn hàng + lịch sử tiền
- `/admin/refunds` Hoàn tiền
- `/admin/payouts` Chi trả theo kỳ
- `/admin/finance` Đối soát
- `/admin/audit-logs` Nhật ký thao tác

Tổng cộng khoảng 29 màn hình.

10. Thiết kế cơ sở dữ liệu (MySQL 8)

Gồm **13 bảng**. Vai trò lưu thẳng trong cột `users.role`; nhân viên gắn với cửa hàng qua `users.store_id`. Tiền dùng `DECIMAL(12,0)` (VNĐ không có số lẻ). Cột in đậm là khoá chính, cột màu cam là khoá ngoại.

users
id
store_id (nhân viên)
role, email (UQ), password_hash, full_name, phone, status, must_change_password

stores
id
owner_id (UQ)
name, phone, address, latitude, longitude, image_url, license_image_url, open/close_time, status, reject_reason, commission_rate, bank_* (bắt buộc)

categories
id
name (UQ), image_url, sort_order

rescue_batches
id
store_id, category_id
title, description, image_url, original_price, rescue_price, quantity, remaining_quantity, pickup_start, pickup_end, status

cart_items
customer_id, batch_id
quantity (1-5), added_at

orders
id
customer_id, store_id, payout_id, cancelled_by, handled_by
order_code (UQ), item_count, total_amount, commission_rate, commission_amount, store_earning, pickup_start, pickup_end, pickup_code (UQ), status, payment_status, cancel_reason, cancelled_at, payment_expires_at, completed_at

order_items
id
order_id, batch_id (UQ cặp)
quantity, unit_price, subtotal

stock_movements
id
batch_id, order_id, actor_id
change_qty (±), reason, remaining_after, note, created_at

payments
id
order_id
amount, txn_ref (UQ), gateway_txn_no, bank_code, response_code, status, paid_at

reviews
id
order_id (UQ), customer_id, store_id
rating (1-5), comment

payouts
id
store_id, created_by, paid_by
payout_code (UQ), period_from, period_to, order_count, gross_amount, commission_amount, net_amount, bank_* (chụp lại), status, transfer_ref, paid_at

refunds
id
order_id (UQ), payment_id, requested_by, processed_by
refund_code (UQ), amount, reason, status, gateway_ref, requested_at, processed_at

audit_logs
id
actor_id (NULL = hệ thống)
action, entity_type, entity_id, amount, detail, created_at

Quan hệ

Quan hệ	Loại	Ghi chú
users (chủ) – stores	1 – 1	Mỗi chủ đúng 1 cửa hàng (`owner_id` UNIQUE).
stores – users (nhân viên)	1 – N	Qua cột `users.store_id`.
stores – rescue_batches	1 – N	Túi thuộc cửa hàng.
orders – order_items – rescue_batches	N – N	1 đơn nhiều túi, 1 túi nằm trong nhiều đơn; `orders.store_id` là cửa hàng chung của mọi túi trong đơn.
users – cart_items – rescue_batches	N – N	Giỏ hàng của khách.
rescue_batches – stock_movements	1 – N	Sổ kho của từng túi.

Quan hệ	Loại	Ghi chú
orders – payments	1 – N	Mỗi lần thanh toán là một giao dịch VNPay.
orders – reviews	1 – 0..1	Mỗi đơn tối đa 1 đánh giá.
stores – payouts	1 – N	Các kỳ chi trả của cửa hàng.
payouts – orders	1 – N	Qua `orders.payout_id`: kỳ chi trả gồm những đơn nào.
orders – refunds	1 – 0..1	Mỗi đơn hoàn tối đa 1 lần; refund trỏ về giao dịch VNPay gốc (`payment_id`).
users – audit_logs	1 – N	Người thực hiện thao tác.

Ràng buộc chính

- `chk_batch_price`: `0 < rescue_price ≤ original_price × 0.6` · `chk_batch_qty`: `0 ≤ remaining_quantity ≤ quantity` · `chk_batch_time`: `pickup_end > pickup_start`.
- `chk_items_money`: `subtotal = unit_price × quantity` · `chk_items_qty`, `chk_cart_qty`: 1-5 cái mỗi loại · `chk_order_pickup`: khung giờ hợp lệ.
- `chk_stock_sign`: LISTED/ADDED/RETURNED phải dương, REMOVED/ORDERED phải âm · `chk_stock_order`: ORDERED/RETURNED bắt buộc có đơn.
- `chk_users_role`: chỉ nhận 4 vai trò; `chk_review_rating`: 1-5.
- `chk_order_money` kiểm tra cả `commission_amount = ROUND(total × rate / 100)`; `chk_order_payout`: chỉ đơn COMPLETED/NO_SHOW mới có `payout_id`.
- `chk_users_staff`: nhân viên bắt buộc có `store_id`; `chk_store_rate`: phí 0-50%; `chk_order_done_time`: COMPLETED/NO_SHOW phải có `completed_at`.
- `chk_payout_money`: `net = gross – commission`; `chk_payout_paid`: PAID phải có mã chuyển khoản; `chk_refund_done`: COMPLETED phải có mã giao dịch hoàn.
- UNIQUE: `users.email`, `orders.order_code`, `orders.pickup_code`, `payments.txn_ref`, `reviews.order_id`, `refunds.order_id`, `payouts.payout_code`, `order_items(order_id, batch_id)`.

Ghi chú khi viết entity JPA

Vấn đề	Cách làm
Khoá chính ghép `cart_items`	Dùng `@IdClass(CartItemId.class)` (2 trường `customerId`, `batchId`) hoặc `@EmbeddedId`; lớp id phải có `equals` / `hashCode`.
`users` ⇄ `stores` tham chiếu lẫn nhau	Lưu theo thứ tự **chủ → cửa hàng → nhân viên**. Trong entity: `Store.owner` là `@OneToOne`, `User.store` là `@ManyToOne` (chỉ nhân viên có). Không dùng cascade giữa hai bên.
Enum trạng thái	`@Enumerated(EnumType.STRING)`, tên hằng trùng đúng chuỗi trong SQL (`PENDING_PAYMENT`, `REFUND_PENDING`...). Không dùng `ORDINAL`.
Tiền	`BigDecimal` (không dùng `double`); làm tròn phí bằng `setScale(0, RoundingMode.HALF_UP)` cho khớp `ROUND()` của MySQL.
Thời gian	`LocalDateTime` cho cột `DATETIME`, `LocalDate` cho `DATE`, `LocalTime` cho `TIME`. Múi giờ cấu hình ở mục 7.6.
Bảng chỉ thêm	`StockMovement`, `AuditLog`: đặt `@Immutable` (Hibernate) và không viết hàm update/delete. CSDL có trigger chặn nếu lỡ gọi.
Tạo bảng	`spring.jpa.hibernate.ddl-auto=validate` (hoặc `none`): bảng tạo từ `foodrescue_schema.sql`, **không** để Hibernate tự tạo (sẽ mất các CHECK, trigger, index).

Vấn đề	Cách làm
Quan hệ lazy	Mọi `@ManyToOne` đặt `fetch = FetchType.LAZY`; API trả DTO, không trả thẳng entity (tránh vòng lặp JSON và lộ `password_hash`).

11. Kế hoạch thực hiện (5 tuần)

11.1 Cách chia việc

- Mỗi việc là **một chức năng trọn vẹn**: Controller + Service + DTO + trang React + test của chức năng đó. Một người làm xong từ đầu đến cuối, không phải chờ người khác.
- Việc ghi Chung sửa file dùng chung (`entity/`, `config/`, `security/`, `routes/`, `services/axios.js`): giao cho một người, làm đầu tuần và merge sớm để các việc khác dựa vào.
- Mỗi việc một nhánh `feature/<mã việc>-<mô tả>` (vd `feature/T2-04-batch-crud`), mở PR vào `develop`; CI xanh + 1 người duyệt mới merge.
- Cột **Người làm** để trống, nhóm tự điền khi chia việc. Mã việc ở đây (T1-01 ...) thay cho mã việc trong lịch 8 tuần cũ.
- Cuối mỗi tuần: merge `develop`, chạy `foodrescue_checks.sql`, demo nội bộ theo mốc của tuần.

Đã xong trước kế hoạch: khung dự án, CSDL 13 bảng, bảo mật JWT + DevAuth, cấu trúc 3 lớp, giỏ hàng, tạo đơn nhiều túi + chống bán vượt, VNPay Sandbox, đơn của tôi, màn hình quầy + giao hàng bằng mã, đơn của cửa hàng + huỷ đơn, tác vụ tự động (24 unit test).

11.2 Tổng quan 5 tuần

Tuần	Thời gian	Trọng tâm	Mốc kiểm tra cuối tuần
Tuần 1	19/10 – 25/10	Đăng nhập thật, khung frontend (router, layout), lưu ảnh	Đăng nhập 4 vai trò bằng JWT, mỗi vai trò vào đúng layout; không còn chọn người dùng giả
Tuần 2	26/10 – 01/11	Cửa hàng, túi giải cứu, danh mục, sổ kho	Chủ đăng ký cửa hàng → admin duyệt → chủ đăng túi có ảnh; sổ kho khớp (C1, C1b)
Tuần 3	02/11 – 08/11	Khách hàng: tìm kiếm, bản đồ, khoảng cách; hoàn thiện luồng mua	**Chạy trọn luồng:** tìm túi gần → giỏ → VNPay → mã nhận hàng → quầy giao hàng
Tuần 4	09/11 – 15/11	Tiền và quản trị: hoàn tiền, chi trả, đối soát, nhân viên, doanh thu, đánh giá	Đối soát C0 = 0 sau kịch bản huỷ đơn + chi trả; admin đủ các trang
Tuần 5	16/11 – 22/11	Kiểm thử, sửa lỗi, giao diện, deploy, báo cáo	Test case mục 13 đạt, CI xanh, demo theo kịch bản mục 14, gắn tag `v1.0`

Ngày ghi theo dự kiến bắt đầu thứ Hai 19/10/2026; nhóm dời lịch thì giữ nguyên thứ tự các tuần.

11.3 Tuần 1 (19/10 – 25/10): Đăng nhập và khung frontend

Mã	Công việc	File chính (backend / frontend)	Xong khi	Người làm
T1-01 Chung	Đưa cấu trúc 3 lớp vào `develop`; nhánh cũ rebase, chuyển file sang đúng package	Toàn repo	Nhánh refactor đã merge, CI xanh, mọi người pull về chạy được	
T1-02	API đăng ký khách, đăng nhập	`AuthController`, `AuthService`, `LoginRequest`, `LoginResponse`, `RegisterRequest`	Đăng nhập trả token + user; sai mật khẩu, tài khoản khoá, email trùng trả đúng mã lỗi	
T1-03	Hồ sơ và đổi mật khẩu	`MeController` (`PUT /api/me`, `/api/me/password`), `AuthService`	Đổi mật khẩu tắt `must_change_password`; sai mật khẩu cũ trả `WRONG_OLD_PASSWORD`	
T1-04	Lưu ảnh tải lên	`FileStorageService`, `config/WebConfig`	Chỉ nhận jpg / png ≤ 5 MB; ảnh mở được tại `/uploads/...`	
T1-05 Chung	Router và layout frontend	`routes/AppRoutes.jsx`, `PrivateRoute.jsx`, `layouts/`, `App.jsx`	Mỗi màn hình có URL riêng; chưa đăng nhập về `/login`; sai vai trò về trang của vai trò	
T1-06	Trang đăng nhập, đăng ký, đổi mật khẩu, hồ sơ	`pages/auth/`, `services/authApi.js`, `hooks/useAuth.js`	Đăng nhập lưu token; bỏ danh sách người dùng giả; mật khẩu tạm bị chuyển tới trang đổi mật khẩu	
T1-07 Chung	Siết cấu hình chỉ dành cho máy dev	`application.yml` (`dev-auth` mặc định `false`), `PaymentController` (`/mock-success`)	Không bật dev-auth thì header `X-Dev-User-Id` và `/mock-success` bị từ chối	
T1-08	Postman collection, kiểm tra Swagger	`docs/postman/`	Mọi API đã có gọi được từ Postman với biến `{{token}}`	

11.4 Tuần 2 (26/10 – 01/11): Cửa hàng, túi, danh mục

Mã	Công việc	File chính (backend / frontend)	Xong khi	Người làm
T2-01	Đăng ký cửa hàng (ảnh giấy phép, ghim bản đồ)	`StoreRegistrationController`, `StoreService`; `pages/store/StoreRegisterPage`	Tạo chủ + cửa hàng `PENDING`; trang đăng ký gọi API thật	
T2-02	Admin duyệt, từ chối, khoá cửa hàng, đặt % phí	`admin/AdminStoreController`, `StoreService`; `pages/admin/AdminStoresPage`	Duyệt xong là `APPROVED`; khoá thì túi bị ẩn (BR-25); ghi log `STORE_*`	
T2-03	Hồ sơ cửa hàng	`store/StoreProfileController`, `StoreService`; `pages/store/StoreProfilePage`	Chủ sửa được thông tin, vị trí, giờ mở cửa, tài khoản ngân hàng	
T2-04	Đăng, sửa, huỷ túi giải cứu	`store/StoreBatchController`, `BatchService`, `BatchRequest`; `pages/store/StoreBatchesPage`, `BatchFormModal`	Giá ≤ 60% (BR-03), khung giờ hợp lệ (BR-04), túi đã có đơn không sửa giá / giờ; bỏ `mocks/storeMockData.js`	
T2-05 Chung	Gom mọi thay đổi số lượng túi vào `StockService`	`StockService` (trừ, trả, điều chỉnh), `OrderService`, `OrderScheduler`	Không còn chỗ nào tự cộng / trừ `remainingQuantity` (BR-23); unit test cũ vẫn xanh	
T2-06	Thêm / bớt túi và sổ kho	`StoreBatchController` (`PATCH .../quantity`, `GET .../stock-movements`); trang sổ kho	`quantity` và `remaining_quantity` cùng cộng / trừ (BR-18); C1, C1b không trả dòng	
T2-07	Hoàn tiền dùng chung, huỷ túi khi đã có đơn	`RefundService.request(order, reason)` (tách từ `OrderService`), `BatchService`	Đơn `CONFIRMED` bị huỷ + chờ hoàn; đơn chưa trả thành `CANCELLED` (BR-24)	
T2-08	Danh mục	`CategoryController`, `admin/AdminCategoryController`, `CategoryService`; `pages/admin/AdminCategoriesPage`	Khách xem theo `sort_order`; admin thêm / sửa / xoá (không xoá danh mục đang có túi)	

11.5 Tuần 3 (02/11 – 08/11): Khách hàng, bản đồ, luồng mua

Mã	Công việc	File chính (backend / frontend)	Xong khi	Người làm
T3-01	Tìm túi theo vị trí, lọc, sắp xếp	`BatchController` (`GET /api/batches`), `CatalogService`, `BatchSearchRepository`	Lọc danh mục, bán kính 1/3/5/10 km; sắp xếp gần nhất / rẻ nhất / sắp hết giờ; chỉ túi của cửa hàng `APPROVED`	
T3-02	Khoảng cách thực tế OSRM	`DistanceService`	1 lần gọi cho ≤ 30 cửa hàng; lỗi hoặc quá 2 giây thì chim bay × 1,3 và `estimated = true`	
T3-03	Trang chủ dùng dữ liệu thật	`pages/customer/HomeCatalogView`, `services/catalogApi.js`, `components/BatchCard`	Bỏ danh sách túi và danh mục giả; mỗi túi hiện "2,3 km · 8 phút"	
T3-04	Bản đồ và định vị	`pages/customer/MapPage`, `hooks/useGeolocation.js`	Marker các cửa hàng có túi; từ chối định vị thì bấm chọn trên bản đồ	
T3-05	Trang cửa hàng công khai	`PublicStoreController`, `CatalogService`; `pages/customer/StorePage`	Thông tin, túi đang bán, đánh giá; cửa hàng bị khoá trả 404	
T3-06	Hoàn thiện luồng mua trên router mới	`CartView`, `PaymentResultView`, `MyOrdersView`; `utils/maps.js`	Giỏ → đặt → VNPay → `/payment/result` → đơn của tôi + QR + nút chỉ đường	
T3-07	Thanh toán về muộn (BR-19)	`PaymentService`, `StockService`, `RefundService`	Tiền về khi đơn đã `EXPIRED`: còn hàng thì xác nhận đơn, hết hàng thì tự tạo hoàn tiền `LATE_PAYMENT`	
T3-08	Màn hình quầy với đăng nhập thật	`pages/store/StorePickupView`, `StoreOrdersView`	Dùng `storeId` từ token; tự tải lại 15 giây	
T3-09	Dữ liệu demo	`database/demo_data.sql`	Khoảng 15 cửa hàng, 30 túi tối nay; chạy `foodrescue_checks.sql` không lỗi	

11.6 Tuần 4 (09/11 – 15/11): Tiền và quản trị

Mã	Công việc	File chính (backend / frontend)	Xong khi	Người làm
T4-01	Admin xác nhận hoàn tiền	`admin/AdminRefundController`, `RefundService`; `pages/admin/AdminRefundsPage`	Bắt buộc mã giao dịch hoàn; refunds `COMPLETED`, đơn `REFUNDED` (BR-16)	
T4-02	Admin đơn hàng	`admin/AdminOrderController`; `pages/admin/AdminOrdersPage`	Lọc, tra mã; chi tiết có lịch sử tiền; admin huỷ đơn `CONFIRMED` + hoàn tiền	
T4-03	Chi trả theo kỳ	`PayoutService`, `admin/AdminPayoutController`; `pages/admin/AdminPayoutsPage`	Xem trước, tạo kỳ, nhập mã chuyển khoản → `PAID`; một đơn không vào 2 kỳ (BR-14, BR-15)	
T4-04	Doanh thu, sao kê, yêu cầu rút tiền	`store/StoreRevenueController`, `RevenueService`; `pages/store/StoreRevenuePage` (Recharts)	Số liệu khớp sao kê từng đơn; yêu cầu rút tiền tạo kỳ `PENDING` (BR-20)	
T4-05	Tài khoản nhân viên	`store/StoreStaffController`, `StaffService`; `pages/store/StoreStaffPage`	Tạo nhân viên với mật khẩu tạm, khoá / mở; lần đầu đăng nhập phải đổi mật khẩu	
T4-06	Admin người dùng	`admin/AdminUserController`, `UserService`; `pages/admin/AdminUsersPage`	Lọc theo email / vai trò, khoá / mở, đặt lại mật khẩu tạm; ghi log `USER_*`	
T4-07	Đối soát và nhật ký	`AdminFinanceController`, `ReconciliationService`, `AdminAuditLogController`; 2 trang admin	7 khoản tiền, chênh lệch = 0 trên dữ liệu mẫu (mục 5.7); lọc nhật ký theo hành động, ngày	
T4-08	Thống kê admin	`AdminDashboardController`, `DashboardService`; `pages/admin/AdminDashboardPage`	Tổng đơn, GMV, phí, túi đã cứu, số cửa hàng / khách, biểu đồ theo ngày	
T4-09	Đánh giá	`ReviewController`, `ReviewService`; `components/ReviewButton`	Chỉ đơn `COMPLETED` của mình, 1 lần (BR-11); điểm hiện ở chi tiết túi và trang cửa hàng	

11.7 Tuần 5 (16/11 – 22/11): Kiểm thử và nộp bài

Mã	Công việc	File chính (backend / frontend)	Xong khi	Người làm
T5-01	Chạy test case mục 13	Toàn hệ thống	Ghi kết quả từng TC; lỗi tạo Issue và sửa trong tuần	
T5-02	Test chống bán vượt	`test/.../OrderConcurrencyTest`	10 luồng cùng đặt túi còn 1 → đúng 1 thành công (TC01)	
T5-03	Bổ sung unit test cho service mới	`test/.../service/`	Mỗi service có test cho quy tắc chính; `mvn verify` xanh trên CI	
T5-04	Kiểm tra dữ liệu sau kịch bản demo	`database/foodrescue_checks.sql`	C0 = 0, C1-C12 không trả dòng nào	
T5-05	Sửa lỗi, làm đẹp giao diện	`pages/` , `components/` , `theme/`	Hiển thị tốt trên laptop và điện thoại; xoá dữ liệu giả, `console.log` , file thừa	
T5-06	Rà bảo mật trước khi nộp	`SecurityConfig` , `controller/store/` , `controller/admin/` , `application.yml`	Mỗi API đúng vai trò; key VNPay, JWT secret không nằm trong Git; dev-auth tắt	
T5-07	Deploy (tuỳ chọn)	Vercel, Render / Railway, MySQL online	Link demo chạy được luồng mua hàng	
T5-08	Báo cáo, slide, video demo	`docs/`	Báo cáo theo tài liệu này; video dự phòng; tập demo theo kịch bản mục 14	
T5-09	Chốt bản nộp	`develop` → `main`	Merge, gắn tag `v1.0` , README cập nhật	

12. Rủi ro

Rủi ro	Cách phòng tránh
Tích hợp VNPay lỗi (sai chữ ký, lệch giờ)	Đã chạy được trên Sandbox và có unit test; so sánh với code mẫu Java của VNPay; xác nhận qua Return URL nên không cần IPN khi chạy local.
OSRM chậm hoặc không truy cập được	Timeout 2 giây và dự phòng khoảng cách ước tính; website vẫn chạy.
Bán vượt số lượng	UPDATE có điều kiện + CHECK trong CSDL; test bằng 2 trình duyệt đặt cùng lúc và test 10 luồng (T5-02).
Frontend và backend lệch API	Chốt danh sách API (mục 8) từ tuần 1; test bằng Postman (T1-08) trước khi nối giao diện.
Nhiều người cùng sửa file dùng chung	Việc Chung giao một người, làm đầu tuần, merge sớm; PR nhỏ, nhãn `shared-change`.
Trễ tiến độ	Ưu tiên luồng mua hàng (mốc tuần 3); việc tuần 4 có thể đơn giản hoá (vd đối soát chỉ hiện bảng tổng, thống kê không có biểu đồ) nếu thiếu thời gian.
Demo lỗi mạng	Chạy local, chuẩn bị sẵn dữ liệu mẫu, quay video demo dự phòng.

13. Kiểm thử

Mã	Kịch bản	Kết quả mong đợi
TC01	2 trình duyệt cùng đặt túi chỉ còn 1	1 người thành công, người kia báo "Túi vừa được giải cứu hết"; số còn lại = 0, túi SOLD_OUT.
TC02	Thanh toán VNPay bằng thẻ test NCB	Quay về trang kết quả, đơn CONFIRMED + PAID, hiện mã nhận hàng.
TC03	Bấm "Huỷ" trên trang VNPay	Đơn EXPIRED, số lượng được trả lại.
TC04	Sửa tham số `vnp_Amount` trên URL trả về	Báo sai chữ ký, đơn không đổi.
TC05	Đặt hàng rồi không thanh toán quá 15 phút	Đơn EXPIRED, số lượng được trả lại.
TC06	Tìm túi quanh vị trí; đổi URL OSRM sang địa chỉ sai	Vẫn ra danh sách, khoảng cách hiện dấu "~".
TC07	Nhân viên cửa hàng A nhập mã đơn của cửa hàng B	Báo "Không tìm thấy đơn", không đổi trạng thái.
TC08	Nhập mã giao hàng trước giờ lấy 1 tiếng	Báo "Chưa tới giờ nhận hàng".
TC09	Tạo túi giá 70.000 / giá gốc 100.000	Báo lỗi "Giá giải cứu phải giảm ít nhất 40%".
TC10	Chủ cửa hàng huỷ đơn đã thanh toán; admin xác nhận hoàn (có và không nhập mã giao dịch)	Đơn CANCELLED + REFUND_PENDING, số lượng trả lại, có 1 bản ghi refunds 45.000đ; không nhập mã thì bị chặn; nhập mã thì refunds COMPLETED, đơn REFUNDED; nhật ký có REFUND_REQUESTED và REFUND_COMPLETED.
TC11	Khách gọi API admin / API cửa hàng	403 Forbidden.
TC12	Đánh giá đơn chưa nhận hoặc đánh giá lần 2	Bị từ chối.
TC13	Tạo kỳ chi trả cho Cơm Tấm 365, kỳ 21-25/09	Xem trước 1 đơn, gross 22.000, phí 3.300, net 18.700; tạo kỳ PENDING; nhập mã chuyển khoản → PAID; chủ cửa hàng thấy kỳ trong sao kê.
TC14	Tạo lại kỳ chi trả trùng khoảng ngày đã chi	Không có đơn đủ điều kiện, không tạo được kỳ (không chi trùng).
TC15	Admin cố huỷ một đơn COMPLETED	Bị chặn (BR-13).
TC16	Mở trang đối soát sau các thao tác trên	Các khoản cộng lại bằng tiền thu, chênh lệch = 0.
TC17	Giả lập thanh toán về sau khi đơn đã EXPIRED (gọi lại URL trả về đã lưu), lần 1 túi còn hàng, lần 2 túi hết	Lần 1: đơn CONFIRMED, số lượng bị trừ lại. Lần 2: đơn CANCELLED + REFUND_PENDING, có refunds lý do LATE_PAYMENT.
TC18	Nhân viên thêm 3 túi rồi bớt 2 túi	`quantity` và `remaining_quantity` cùng tăng 3 rồi giảm 2; bớt quá số còn lại bị chặn; C1 vẫn trả về 0 dòng.
TC19	Chủ Cơm Tấm 365 bấm "Yêu cầu rút tiền" 2 lần	Lần 1 tạo kỳ PENDING 18.700đ; lần 2 bị chặn vì đang có kỳ PENDING (BR-20).
TC20	Chạy `foodrescue_checks.sql` sau toàn bộ test	C1-C12 trả về 0 dòng, C0 chênh lệch = 0.
TC21	Giỏ đang có túi Sweet Paris, thêm túi của An Phát	API trả lỗi "giỏ khác cửa hàng"; React hỏi xoá giỏ cũ; đồng ý thì giỏ chỉ còn túi An Phát.
TC22	Đặt đơn 2 loại túi, trong đó 1 loại vừa hết	Báo đúng túi bị thiếu; không túi nào bị trừ; không tạo đơn; giỏ vẫn còn nguyên.

Mã	Kịch bản	Kết quả mong đợi
TC23	Đăng túi 10 cái, thêm 2, bớt 1, khách đặt 3, đơn hết hạn	Sổ kho có 5 dòng LISTED +10, ADDED +2, REMOVED −1, ORDERED −3, RETURNED +3; còn lại 11, quantity 11; C1b sạch.
TC24	Admin đặt lại mật khẩu cho khách, khách đăng nhập bằng mật khẩu tạm	Bị chuyển tới trang đổi mật khẩu; gọi API khác trả 403 cho đến khi đổi; nhật ký có USER_PASSWORD_RESET.
TC25	Chạy `UPDATE` hoặc `DELETE` trên `stock_movements` / `audit_logs`	MySQL báo lỗi (trigger), dữ liệu không đổi.
TC26	Chủ cửa hàng huỷ túi đang có 1 đơn đã trả tiền và 1 đơn chưa trả tiền	Đơn đã trả: CANCELLED + REFUND_PENDING, có refunds BATCH_CANCELLED. Đơn chưa trả: CANCELLED + UNPAID. Cả hai trả kho (RETURNED); C1-C12 vẫn sạch.

14. Tài khoản và kịch bản demo

14.1 Tài khoản mẫu (mật khẩu: `123456`)

Email	Vai trò	Tình trạng dữ liệu
admin@foodrescue.vn	Admin	1 cửa hàng chờ duyệt (GreenMart); 1 yêu cầu hoàn tiền 45.000đ đang chờ; An Phát đã được chi 170.000đ (kỳ PO-1-20260924); Cơm Tấm 365 còn 18.700đ chưa chi.
owner.anphat@gmail.com	Chủ cửa hàng	Có 2 đơn đã giao, đánh giá 5 và 4 sao, đã được chi 170.000đ (có mã chuyển khoản).
owner.sweetparis@gmail.com	Chủ cửa hàng	Có đơn đang chờ lấy, đơn đã huỷ, đơn chờ thanh toán.
owner.comtam365@gmail.com	Chủ cửa hàng	Có đơn khách không đến lấy (NO_SHOW), 18.700đ chưa được chi, dùng để demo yêu cầu rút tiền.
owner.greenmart@gmail.com	Chủ cửa hàng	Cửa hàng đang chờ duyệt.
staff.anphat@gmail.com	Nhân viên	Nhân viên An Phát.
staff.sweetparis@gmail.com	Nhân viên	Nhân viên Sweet Paris.
an.malyhoang@gmail.com	Khách hàng	Có đơn 2 loại túi (87.000đ) đang chờ lấy tại Sweet Paris, mã `H2R9W7`.
linh.tranhoang@gmail.com	Khách hàng	Giỏ hàng đang có 2 loại túi của Sweet Paris, dùng để demo đặt hàng từ giỏ.

14.2 Kịch bản demo (khoảng 8 phút)

1. **Admin** duyệt cửa hàng GreenMart (xem giấy phép và vị trí trên bản đồ).
2. **Chủ GreenMart** đăng "Túi rau củ cuối ngày"; thử nhập giá chỉ giảm 30% để thấy bị chặn.
3. **Khách** mở bản đồ quanh tôi: thấy các túi kèm "x km · y phút"; thêm 2 loại túi của cùng quán vào giỏ, đặt hàng và thanh toán VNPay bằng thẻ test.
4. **Nhân viên** mở màn hình quầy, thấy đơn mới, nhập mã khách đọc và bấm "Đã giao".
5. **Khách** đánh giá 5 sao.
6. Hai trình duyệt cùng đặt túi còn 1 để chứng minh không bán vượt.
7. **Chủ Cơm Tấm 365** xem doanh thu và sao kê từng đơn, thêm 3 túi cho túi đang bán, bấm "Yêu cầu rút tiền"; tạo tài khoản nhân viên.

8. **Admin** xác nhận hoàn tiền đơn bị huỷ (nhập mã giao dịch hoàn), xác nhận yêu cầu rút tiền của Cơm Tấm 365 bằng mã chuyển khoản, mở trang đối soát thấy chênh lệch = 0, xem nhật ký thao tác.

Phụ lục A. Script cơ sở dữ liệu (MySQL 8)

File đi kèm: `foodrescue_schema.sql` (13 bảng, 25 ràng buộc CHECK, 4 trigger + dữ liệu mẫu; đã chạy thử trên MySQL 8.0.40). File kiểm tra dữ liệu: `foodrescue_checks.sql` (mục 5.10). Lưu ý: script bắt đầu bằng `DROP DATABASE IF EXISTS foodrescuedb`.

```
-- ========================================================================
--  FOODRESCUE - Hệ thống giải cứu thực phẩm cuối ngày (bản rút gọn)
--  Script CSDL v5.4 - MySQL 8.0.16+ (utf8mb4) - 13 bảng
--  v5  : quản lý tiền rõ ràng (refunds, payouts theo kỳ gắn với đơn, tỉ lệ phí lưu theo đơn),
--        nhật ký thao tác (audit_logs)
--  v5.1: tỉ lệ phí theo cửa hàng, cửa hàng tự yêu cầu rút tiền, 1 chủ = 1 cửa hàng,
--        hoàn tiền khi thanh toán về muộn, ràng buộc nhân viên - cửa hàng
--  v5.2: giỏ hàng (cart_items), đơn nhiều túi (order_items), sổ kho (stock_movements)
--  v5.3: múi giờ +07:00, admin đặt lại mật khẩu tạm (must_change_password),
--        trigger chặn sửa/xoá sổ kho và nhật ký
--  v5.4: danh mục có ảnh và thứ tự hiển thị (thanh danh mục trang chủ)
--  Múi giờ: mọi cột DATETIME là giờ Việt Nam. Backend phải đặt cùng múi giờ
--        (JDBC serverTimezone=Asia/Ho_Chi_Minh, JVM -Duser.timezone=Asia/Ho_Chi_Minh).
--  Mật khẩu mặc định của mọi tài khoản mẫu: 123456 (BCrypt cost 10)
--  CẢNH BÁO: script xoá và tạo lại database foodrescuedb
-- ========================================================================
DROP DATABASE IF EXISTS foodrescuedb;
CREATE DATABASE foodrescuedb CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE foodrescuedb;
SET time_zone = '+07:00';   -- NOW() trong phiên này theo giờ Việt Nam

-- ------------------------------------------------------------------------
-- 1. Người dùng (4 vai trò)
-- ------------------------------------------------------------------------
CREATE TABLE users (
    id             BIGINT AUTO_INCREMENT PRIMARY KEY,
    role           VARCHAR(20)  NOT NULL,                  -- ADMIN, STORE_OWNER, STORE_STAFF, CUSTOMER
    email          VARCHAR(100) NOT NULL UNIQUE,
    password_hash  VARCHAR(255) NOT NULL,
    full_name      VARCHAR(100) NOT NULL,
    phone          VARCHAR(20)  NOT NULL,
    store_id       BIGINT,                                 -- chỉ dùng cho STORE_STAFF
    status         VARCHAR(20)  NOT NULL DEFAULT 'ACTIVE', -- ACTIVE, LOCKED
    must_change_password BOOLEAN NOT NULL DEFAULT FALSE,   -- TRUE: mật khẩu tạm (nhân viên mới / admin đặt lại)
    created_at     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_users_role  CHECK (role IN ('ADMIN', 'STORE_OWNER', 'STORE_STAFF', 'CUSTOMER')),
    -- nhân viên bắt buộc thuộc 1 cửa hàng; vai trò khác không có store_id
    CONSTRAINT chk_users_staff CHECK ((role = 'STORE_STAFF') = (store_id IS NOT NULL))
);

-- ------------------------------------------------------------------------
-- 2. Cửa hàng
-- ------------------------------------------------------------------------
CREATE TABLE stores (
    id                BIGINT AUTO_INCREMENT PRIMARY KEY,
    owner_id          BIGINT        NOT NULL UNIQUE,   -- 1 chủ = 1 cửa hàng
    name              VARCHAR(150)  NOT NULL,
    description       TEXT,
    phone             VARCHAR(20)   NOT NULL,
    address           VARCHAR(255)  NOT NULL,
    latitude          DECIMAL(10,8) NOT NULL,        -- chủ quán ghim trên bản đồ
    longitude         DECIMAL(11,8) NOT NULL,
    image_url         VARCHAR(255),
    license_image_url VARCHAR(255)  NOT NULL,        -- ảnh giấy phép VSATTP
    open_time         TIME          NOT NULL,
    close_time        TIME          NOT NULL,
    status            VARCHAR(20)   NOT NULL DEFAULT 'PENDING',  -- PENDING, APPROVED, REJECTED, LOCKED
    reject_reason     VARCHAR(255),
    commission_rate   DECIMAL(5,2)  NOT NULL DEFAULT 15.00,  -- % phí nền tảng, admin đặt; đơn mới copy sang orders
    bank_name         VARCHAR(100)  NOT NULL,        -- tài khoản nhận tiền chi trả (bắt buộc)
    bank_account_no   VARCHAR(30)   NOT NULL,
    bank_account_name VARCHAR(100)  NOT NULL,
    created_at        DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_stores_owner FOREIGN KEY (owner_id) REFERENCES users(id),
    CONSTRAINT chk_store_time  CHECK (close_time > open_time),
    CONSTRAINT chk_store_rate  CHECK (commission_rate BETWEEN 0 AND 50)
);
CREATE INDEX idx_stores_status_geo ON stores(status, latitude, longitude);

ALTER TABLE users ADD CONSTRAINT fk_users_store FOREIGN KEY (store_id) REFERENCES stores(id);
```

```
-- ---------------------------------------------------------------------
-- 3. Danh mục
-- ---------------------------------------------------------------------
CREATE TABLE categories (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL UNIQUE,
    image_url   VARCHAR(255),                   -- ảnh vuông nền trong, hiện trên thanh danh mục
    sort_order  INT          NOT NULL DEFAULT 0  -- nhỏ đứng trước
);

-- ---------------------------------------------------------------------
-- 4. Túi giải cứu (hàng đăng bán = tồn kho hiện tại)
-- ---------------------------------------------------------------------
CREATE TABLE rescue_batches (
    id                BIGINT AUTO_INCREMENT PRIMARY KEY,
    store_id          BIGINT        NOT NULL,
    category_id       BIGINT        NOT NULL,
    title             VARCHAR(150)  NOT NULL,
    description       TEXT,
    image_url         VARCHAR(255),
    original_price    DECIMAL(12,0) NOT NULL,
    rescue_price      DECIMAL(12,0) NOT NULL,
    quantity          INT           NOT NULL,      -- tổng số túi đã đưa lên app = đã bán + còn lại
    remaining_quantity  INT         NOT NULL,      -- còn lại để bán
    pickup_start      DATETIME      NOT NULL,
    pickup_end        DATETIME      NOT NULL,
    status            VARCHAR(20)   NOT NULL DEFAULT 'AVAILABLE',  -- AVAILABLE, SOLD_OUT, EXPIRED, CANCELLED
    created_at        DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_batch_store    FOREIGN KEY (store_id)    REFERENCES stores(id),
    CONSTRAINT fk_batch_category FOREIGN KEY (category_id) REFERENCES categories(id),
    CONSTRAINT chk_batch_price   CHECK (rescue_price > 0 AND rescue_price <= original_price * 0.6),
    CONSTRAINT chk_batch_qty     CHECK (quantity >= 0 AND remaining_quantity BETWEEN 0 AND quantity),
    CONSTRAINT chk_batch_time    CHECK (pickup_end > pickup_start)
);
CREATE INDEX idx_batch_listing ON rescue_batches(status, pickup_end);

-- ---------------------------------------------------------------------
-- 5. Kỳ chi trả cho cửa hàng: admin tạo theo kỳ, HOẶC chủ cửa hàng bấm "Yêu cầu rút tiền"
--    (created_by = người tạo). Gom các đơn chưa chi trong kỳ.
-- ---------------------------------------------------------------------
CREATE TABLE payouts (
    id                BIGINT AUTO_INCREMENT PRIMARY KEY,
    payout_code       VARCHAR(30)   NOT NULL UNIQUE,     -- PO-<storeId>-yyyyMMdd
    store_id          BIGINT        NOT NULL,
    period_from       DATE          NOT NULL,            -- kỳ đối soát (theo ngày chốt đơn)
    period_to         DATE          NOT NULL,
    order_count       INT           NOT NULL,
    gross_amount      DECIMAL(14,0) NOT NULL,            -- tổng khách đã trả của các đơn trong kỳ
    commission_amount DECIMAL(14,0) NOT NULL,            -- tổng phí nền tảng
    net_amount        DECIMAL(14,0) NOT NULL,            -- số tiền chuyển cho cửa hàng
    bank_name         VARCHAR(100)  NOT NULL,            -- chụp lại tài khoản tại thời điểm tạo
    bank_account_no   VARCHAR(30)   NOT NULL,
    bank_account_name VARCHAR(100)  NOT NULL,
    status            VARCHAR(20)   NOT NULL DEFAULT 'PENDING',  -- PENDING, PAID, CANCELLED
    transfer_ref      VARCHAR(100),                      -- mã giao dịch chuyển khoản ngân hàng
    note              VARCHAR(255),
    created_by        BIGINT        NOT NULL,            -- admin, hoặc chủ cửa hàng (yêu cầu rút tiền)
    created_at        DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    paid_by           BIGINT,
    paid_at           DATETIME,
    CONSTRAINT fk_payouts_store   FOREIGN KEY (store_id)   REFERENCES stores(id),
    CONSTRAINT fk_payouts_creator FOREIGN KEY (created_by) REFERENCES users(id),
    CONSTRAINT fk_payouts_payer   FOREIGN KEY (paid_by)    REFERENCES users(id),
    CONSTRAINT chk_payout_money   CHECK (order_count > 0 AND net_amount > 0
                                         AND net_amount = gross_amount - commission_amount),
    CONSTRAINT chk_payout_period  CHECK (period_to >= period_from),
    CONSTRAINT chk_payout_paid    CHECK (status <> 'PAID' OR (paid_at IS NOT NULL AND transfer_ref IS NOT NULL))
);

-- ---------------------------------------------------------------------
-- 6. Giỏ hàng (lưu trong CSDL; chỉ chứa túi của 1 cửa hàng - kiểm tra ở service)
--    Giỏ hàng KHÔNG giữ hàng; hàng chỉ bị trừ khi đặt đơn.
-- ---------------------------------------------------------------------
CREATE TABLE cart_items (
    customer_id  BIGINT   NOT NULL,
    batch_id     BIGINT   NOT NULL,
    quantity     INT      NOT NULL,
    added_at     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (customer_id, batch_id),
    CONSTRAINT fk_cart_customer FOREIGN KEY (customer_id) REFERENCES users(id),
    CONSTRAINT fk_cart_batch    FOREIGN KEY (batch_id)    REFERENCES rescue_batches(id),
```

```
    CONSTRAINT chk_cart_qty     CHECK (quantity BETWEEN 1 AND 5)
);

-- ---------------------------------------------------------------------
-- 7. Đơn hàng (1 đơn = 1 cửa hàng, nhiều túi)
-- ---------------------------------------------------------------------
CREATE TABLE orders (
    id                 BIGINT AUTO_INCREMENT PRIMARY KEY,
    order_code         VARCHAR(30)   NOT NULL UNIQUE,
    customer_id        BIGINT        NOT NULL,
    store_id           BIGINT        NOT NULL,
    item_count         INT           NOT NULL,     -- tổng số túi trong đơn
    total_amount       DECIMAL(12,0) NOT NULL,     -- = tổng order_items.subtotal (khách trả)
    commission_rate    DECIMAL(5,2)  NOT NULL,     -- tỉ lệ phí của cửa hàng tại thời điểm đặt, vd 15.00
    commission_amount  DECIMAL(12,0) NOT NULL,     -- = ROUND(total_amount * commission_rate / 100)
    store_earning      DECIMAL(12,0) NOT NULL,     -- = total_amount - commission_amount
    pickup_start       DATETIME      NOT NULL,     -- khung giờ lấy của đơn = phần giao nhau
    pickup_end         DATETIME      NOT NULL,     --   khung giờ của các túi trong đơn
    pickup_code        VARCHAR(10)   NOT NULL UNIQUE,
    status             VARCHAR(20)   NOT NULL DEFAULT 'PENDING_PAYMENT',
        -- PENDING_PAYMENT, CONFIRMED, COMPLETED, CANCELLED, EXPIRED, NO_SHOW
        -- CANCELLED + UNPAID: khách tự huỷ đơn chưa trả tiền; CANCELLED + REFUND_*: đã trả tiền rồi bị huỷ
    payment_status     VARCHAR(20)   NOT NULL DEFAULT 'UNPAID',
        -- UNPAID, PAID, REFUND_PENDING, REFUNDED
    payout_id          BIGINT,                     -- kỳ chi trả đã gom đơn này (NULL = chưa chi)
    cancel_reason      VARCHAR(255),
    cancelled_by       BIGINT,
    cancelled_at       DATETIME,
    handled_by         BIGINT,                     -- người giao hàng (chủ/nhân viên)
    payment_expires_at DATETIME      NOT NULL,     -- link VNPay hết hạn sau 10 phút
    completed_at       DATETIME,                   -- thời điểm chốt đơn: giao hàng (COMPLETED) hoặc NO_SHOW
    created_at         DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_orders_customer  FOREIGN KEY (customer_id)  REFERENCES users(id),
    CONSTRAINT fk_orders_store     FOREIGN KEY (store_id)     REFERENCES stores(id),
    CONSTRAINT fk_orders_payout    FOREIGN KEY (payout_id)    REFERENCES payouts(id),
    CONSTRAINT fk_orders_canceller FOREIGN KEY (cancelled_by) REFERENCES users(id),
    CONSTRAINT fk_orders_handler   FOREIGN KEY (handled_by)   REFERENCES users(id),
    CONSTRAINT chk_order_items     CHECK (item_count > 0),
    CONSTRAINT chk_order_money     CHECK (total_amount > 0
                                       AND commission_amount = ROUND(total_amount * commission_rate / 100)
                                       AND store_earning = total_amount - commission_amount),
    CONSTRAINT chk_order_pickup    CHECK (pickup_end > pickup_start),
    -- chỉ đơn đã hoàn tất / không đến lấy mới được đưa vào kỳ chi trả
    CONSTRAINT chk_order_payout    CHECK (payout_id IS NULL OR status IN ('COMPLETED', 'NO_SHOW')),
    CONSTRAINT chk_order_done_time CHECK ((status IN ('COMPLETED', 'NO_SHOW')) = (completed_at IS NOT NULL))
);
CREATE INDEX idx_orders_store    ON orders(store_id, status, created_at);
CREATE INDEX idx_orders_customer ON orders(customer_id, created_at);
CREATE INDEX idx_orders_payout   ON orders(store_id, payout_id, status, completed_at);

-- ---------------------------------------------------------------------
-- 8. Chi tiết đơn (các túi trong đơn, giá chụp lại lúc mua)
-- ---------------------------------------------------------------------
CREATE TABLE order_items (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    order_id    BIGINT        NOT NULL,
    batch_id    BIGINT        NOT NULL,
    quantity    INT           NOT NULL,
    unit_price  DECIMAL(12,0) NOT NULL,       -- giá giải cứu tại thời điểm mua
    subtotal    DECIMAL(12,0) NOT NULL,       -- = unit_price * quantity
    CONSTRAINT fk_items_order  FOREIGN KEY (order_id) REFERENCES orders(id),
    CONSTRAINT fk_items_batch  FOREIGN KEY (batch_id) REFERENCES rescue_batches(id),
    CONSTRAINT uq_items_batch  UNIQUE (order_id, batch_id),
    CONSTRAINT chk_items_qty   CHECK (quantity BETWEEN 1 AND 5),
    CONSTRAINT chk_items_money CHECK (subtotal = unit_price * quantity)
);

-- ---------------------------------------------------------------------
-- 9. Sổ kho: mỗi lần số lượng túi thay đổi ghi 1 dòng (chỉ thêm, không sửa/xoá)
--    quantity  = tổng change_qty của LISTED + ADDED + REMOVED
--    remaining = tổng change_qty của mọi dòng
-- ---------------------------------------------------------------------
CREATE TABLE stock_movements (
    id              BIGINT AUTO_INCREMENT PRIMARY KEY,
    batch_id        BIGINT       NOT NULL,
    change_qty      INT          NOT NULL,        -- dương = tăng, âm = giảm
    reason          VARCHAR(20)  NOT NULL,
        -- LISTED  : đăng túi (+ số lượng ban đầu)
        -- ADDED   : cửa hàng thêm túi (+)
        -- REMOVED : bớt túi, vd khách tại quầy mua mất (−)
        -- ORDERED : khách đặt đơn (−)
```

```
        -- RETURNED: đơn hết hạn / huỷ, trả lại (+)
    order_id        BIGINT,                     -- với ORDERED / RETURNED
    actor_id        BIGINT,                     -- NULL = hệ thống
    remaining_after INT          NOT NULL,      -- số còn lại sau thay đổi
    note            VARCHAR(255),
    created_at      DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_stock_batch FOREIGN KEY (batch_id) REFERENCES rescue_batches(id),
    CONSTRAINT fk_stock_order FOREIGN KEY (order_id) REFERENCES orders(id),
    CONSTRAINT fk_stock_actor FOREIGN KEY (actor_id) REFERENCES users(id),
    CONSTRAINT chk_stock_nonzero CHECK (change_qty <> 0),
    CONSTRAINT chk_stock_sign CHECK (
        (reason IN ('LISTED', 'ADDED', 'RETURNED') AND change_qty > 0) OR
        (reason IN ('REMOVED', 'ORDERED')          AND change_qty < 0)),
    CONSTRAINT chk_stock_order CHECK ((reason IN ('ORDERED', 'RETURNED')) = (order_id IS NOT NULL)),
    CONSTRAINT chk_stock_after CHECK (remaining_after >= 0)
);
CREATE INDEX idx_stock_batch ON stock_movements(batch_id, created_at);

-- Sổ kho chỉ được thêm, không được sửa / xoá
DELIMITER $
CREATE TRIGGER trg_stock_no_update BEFORE UPDATE ON stock_movements FOR EACH ROW
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'stock_movements chỉ được thêm, không được sửa'$
CREATE TRIGGER trg_stock_no_delete BEFORE DELETE ON stock_movements FOR EACH ROW
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'stock_movements chỉ được thêm, không được xoá'$
DELIMITER ;

-- ------------------------------------------------------------------------
-- 10. Giao dịch thanh toán VNPay
-- ------------------------------------------------------------------------
CREATE TABLE payments (
    id              BIGINT AUTO_INCREMENT PRIMARY KEY,
    order_id        BIGINT        NOT NULL,
    amount          DECIMAL(12,0) NOT NULL,
    txn_ref         VARCHAR(64)   NOT NULL UNIQUE,   -- vnp_TxnRef
    gateway_txn_no  VARCHAR(64),                     -- vnp_TransactionNo (cần khi hoàn tiền)
    bank_code       VARCHAR(20),
    response_code   VARCHAR(10),                     -- vnp_ResponseCode
    status          VARCHAR(20)   NOT NULL DEFAULT 'PENDING',  -- PENDING, SUCCESS, FAILED
    paid_at         DATETIME,
    created_at      DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_payments_order FOREIGN KEY (order_id) REFERENCES orders(id)
);

-- ------------------------------------------------------------------------
-- 11. Hoàn tiền cho khách
-- ------------------------------------------------------------------------
CREATE TABLE refunds (
    id              BIGINT AUTO_INCREMENT PRIMARY KEY,
    refund_code     VARCHAR(30)   NOT NULL UNIQUE,   -- RF-<orderCode>
    order_id        BIGINT        NOT NULL UNIQUE,   -- mỗi đơn hoàn tối đa 1 lần (toàn bộ)
    payment_id      BIGINT        NOT NULL,          -- giao dịch VNPay gốc
    amount          DECIMAL(12,0) NOT NULL,          -- = orders.total_amount
    reason          VARCHAR(30)   NOT NULL,          -- STORE_CANCELLED, BATCH_CANCELLED, ADMIN_CANCELLED, LATE_PAYMENT
    status          VARCHAR(20)   NOT NULL DEFAULT 'PENDING',  -- PENDING, COMPLETED
    gateway_ref     VARCHAR(100),                    -- mã giao dịch hoàn trên VNPay merchant
    note            VARCHAR(255),
    requested_by    BIGINT,                          -- người huỷ đơn; NULL = hệ thống (LATE_PAYMENT)
    requested_at    DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    processed_by    BIGINT,                          -- admin xác nhận đã hoàn
    processed_at    DATETIME,
    CONSTRAINT fk_refunds_order     FOREIGN KEY (order_id)     REFERENCES orders(id),
    CONSTRAINT fk_refunds_payment   FOREIGN KEY (payment_id)   REFERENCES payments(id),
    CONSTRAINT fk_refunds_requester FOREIGN KEY (requested_by) REFERENCES users(id),
    CONSTRAINT fk_refunds_processor FOREIGN KEY (processed_by) REFERENCES users(id),
    CONSTRAINT chk_refund_amount    CHECK (amount > 0),
    CONSTRAINT chk_refund_done      CHECK (status <> 'COMPLETED' OR (processed_at IS NOT NULL AND gateway_ref IS NOT NULL))
);

-- ------------------------------------------------------------------------
-- 12. Đánh giá (1 đơn 1 đánh giá, đánh giá cửa hàng)
-- ------------------------------------------------------------------------
CREATE TABLE reviews (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    order_id    BIGINT   NOT NULL UNIQUE,
    customer_id BIGINT   NOT NULL,
    store_id    BIGINT   NOT NULL,
    rating      TINYINT  NOT NULL,
    comment     TEXT,
    created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_reviews_order    FOREIGN KEY (order_id)    REFERENCES orders(id),
    CONSTRAINT fk_reviews_customer FOREIGN KEY (customer_id) REFERENCES users(id),
```

```
    CONSTRAINT fk_reviews_store    FOREIGN KEY (store_id)    REFERENCES stores(id),
    CONSTRAINT chk_review_rating   CHECK (rating BETWEEN 1 AND 5)
);

-- ---------------------------------------------------------------------
-- 13. Nhật ký thao tác (chỉ thêm, không sửa/xoá)
-- ---------------------------------------------------------------------
CREATE TABLE audit_logs (
    id           BIGINT AUTO_INCREMENT PRIMARY KEY,
    actor_id     BIGINT,                          -- NULL = hệ thống (VNPay, tác vụ tự động)
    action       VARCHAR(40)  NOT NULL,
        -- PAYMENT_SUCCESS, ORDER_CANCELLED, ORDER_EXPIRED, ORDER_COMPLETED, ORDER_NO_SHOW,
        -- REFUND_REQUESTED, REFUND_COMPLETED, PAYOUT_CREATED, PAYOUT_PAID, PAYOUT_CANCELLED,
        -- BATCH_CANCELLED, STORE_APPROVED, STORE_REJECTED, STORE_LOCKED, STORE_RATE_CHANGED,
        -- USER_LOCKED, USER_UNLOCKED, USER_PASSWORD_RESET
    entity_type  VARCHAR(20)  NOT NULL,           -- ORDER, REFUND, PAYOUT, STORE, USER, BATCH
    entity_id    BIGINT       NOT NULL,
    amount       DECIMAL(14,0),                   -- số tiền liên quan (nếu có)
    detail       VARCHAR(500),
    created_at   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_audit_actor FOREIGN KEY (actor_id) REFERENCES users(id)
);
CREATE INDEX idx_audit_entity ON audit_logs(entity_type, entity_id);
CREATE INDEX idx_audit_time   ON audit_logs(created_at);

-- Nhật ký chỉ được thêm, không được sửa / xoá
DELIMITER $
CREATE TRIGGER trg_audit_no_update BEFORE UPDATE ON audit_logs FOR EACH ROW
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'audit_logs chỉ được thêm, không được sửa'$
CREATE TRIGGER trg_audit_no_delete BEFORE DELETE ON audit_logs FOR EACH ROW
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'audit_logs chỉ được thêm, không được xoá'$
DELIMITER ;

-- ======================================================================
--  DỮ LIỆU MẪU ("hôm nay" = 2026-09-25)
-- ======================================================================
SET @pw = '$2a$10$Z9WS6Xe2NDqRK7M4rKoyV.uxSnXxsllWiVAbVXxa4tvQyn1nxeNni';  -- 123456
INSERT INTO users (id, role, email, password_hash, full_name, phone, status) VALUES
( 1, 'ADMIN',       'admin@foodrescue.vn',        @pw, 'Quản Trị Viên',   '0909000001', 'ACTIVE'),
( 2, 'STORE_OWNER', 'owner.anphat@gmail.com',     @pw, 'Nguyễn Văn Phát', '0912345671', 'ACTIVE'),
( 3, 'STORE_OWNER', 'owner.sweetparis@gmail.com', @pw, 'Lê Thị Thu Thảo', '0912345672', 'ACTIVE'),
( 4, 'STORE_OWNER', 'owner.comtam365@gmail.com',  @pw, 'Trần Văn Cali',   '0912345673', 'ACTIVE'),
( 5, 'STORE_OWNER', 'owner.greenmart@gmail.com',  @pw, 'Võ Thị Hạnh',     '0912345674', 'ACTIVE'),
( 8, 'CUSTOMER',    'an.malyhoang@gmail.com',     @pw, 'Ma Lý Hoàng Ân',  '0987654321', 'ACTIVE'),
( 9, 'CUSTOMER',    'linh.tranhoang@gmail.com',   @pw, 'Trần Hoàng Linh', '0987654322', 'ACTIVE'),
(10, 'CUSTOMER',    'bao.lequoc@gmail.com',       @pw, 'Lê Quốc Bảo',     '0987654323', 'ACTIVE'),
(11, 'CUSTOMER',    'thu.phamminh@gmail.com',     @pw, 'Phạm Minh Thư',   '0987654324', 'ACTIVE');

INSERT INTO stores (id, owner_id, name, description, phone, address, latitude, longitude, license_image_url,
                    open_time, close_time, status, commission_rate, bank_name, bank_account_no, bank_account_name) VALUES
(1, 2, 'Bánh Mì Tươi & Hamburger An Phát', 'Bánh mì, sandwich, hamburger làm trong ngày.', '02838300001',
    '105 An Dương Vương, Phường 8, Quận 5, TP.HCM', 10.75837200, 106.67123900, '/uploads/license-1.jpg',
    '06:30:00', '21:30:00', 'APPROVED', 15.00, 'Vietcombank', '0071000123456', 'NGUYEN VAN PHAT'),
(2, 3, 'Sweet Paris Bakery & Coffee', 'Bánh ngọt kiểu Pháp và cà phê.', '02838300002',
    '45 Lê Duẩn, Bến Nghé, Quận 1, TP.HCM', 10.78125400, 106.69894200, '/uploads/license-2.jpg',
    '07:00:00', '22:00:00', 'APPROVED', 15.00, 'Techcombank', '19031234567890', 'LE THI THU THAO'),
(3, 4, 'Cơm Tấm Truyền Thống Sài Gòn 365', 'Cơm tấm sườn bì chả truyền thống.', '02838300003',
    '215 Điện Biên Phủ, Phường 15, Bình Thạnh, TP.HCM', 10.79634100, 106.70921800, '/uploads/license-3.jpg',
    '10:00:00', '21:00:00', 'APPROVED', 15.00, 'ACB', '2468101214', 'TRAN VAN CALI'),
(4, 5, 'GreenMart Rau Củ Sạch', 'Rau củ quả hữu cơ.', '02838300004',
    '12 Nguyễn Thị Minh Khai, Đa Kao, Quận 1, TP.HCM', 10.78756000, 106.69980000, '/uploads/license-4.jpg',
    '07:00:00', '21:00:00', 'PENDING', 10.00, 'MB Bank', '0123456789', 'VO THI HANH');

-- Nhân viên chèn sau cửa hàng (vì bắt buộc có store_id)
INSERT INTO users (id, role, email, password_hash, full_name, phone, store_id, status) VALUES
( 6, 'STORE_STAFF', 'staff.anphat@gmail.com',     @pw, 'Đỗ Minh Khoa',    '0933000001', 1, 'ACTIVE'),   -- cả 2 đã đổi mật khẩu
( 7, 'STORE_STAFF', 'staff.sweetparis@gmail.com', @pw, 'Huỳnh Ngọc Mai',  '0933000002', 2, 'ACTIVE');

INSERT INTO categories (id, name, image_url, sort_order) VALUES
(1, 'Bánh mì & Sandwich', '/uploads/categories/banh-mi.png',   1),
(2, 'Bánh ngọt',          '/uploads/categories/banh-ngot.png',  2),
(3, 'Cơm & Món chính',    '/uploads/categories/com.png',        3),
(4, 'Rau củ & Trái cây',  '/uploads/categories/rau-cu.png',     4),
(5, 'Đồ uống',            '/uploads/categories/do-uong.png',    5),
(6, 'Tạp hoá',            '/uploads/categories/tap-hoa.png',    6);

INSERT INTO rescue_batches (id, store_id, category_id, title, description, original_price, rescue_price,
       quantity, remaining_quantity, pickup_start, pickup_end, status, created_at) VALUES
(1, 1, 1, 'Túi Bánh Mì Bất Ngờ', '2 bánh mì hoa cúc mini + 1 sandwich xúc xích trong ngày.',
    80000, 30000, 10, 8, '2026-09-25 20:30:00', '2026-09-25 21:30:00', 'AVAILABLE', '2026-09-25 09:00:00'),
(2, 1, 1, 'Combo Hamburger Cuối Giờ', '2 burger bò/gà giữ nóng tại quầy.',
```

```
    95000, 40000, 5, 0, '2026-09-23 20:30:00', '2026-09-23 21:30:00', 'SOLD_OUT', '2026-09-23 17:00:00'),
(3, 2, 2, 'Surprise Box Bánh Pháp', 'Ngẫu nhiên 3 loại: croissant phô mai, tart trứng, donut kem.',
    120000, 45000, 15, 14, '2026-09-25 21:00:00', '2026-09-25 22:00:00', 'AVAILABLE', '2026-09-25 08:00:00'),
(4, 2, 2, 'Bánh Kem Lát Mini', 'Bánh kem bắp và tiramisu lát, bảo quản tủ mát.',
    110000, 42000, 8, 6, '2026-09-25 21:00:00', '2026-09-25 22:00:00', 'AVAILABLE', '2026-09-25 08:05:00'),
(5, 3, 3, 'Suất Cơm Sườn Bì Chả', 'Cơm nóng đóng hộp gồm sườn nướng, bì, chả trứng.',
    65000, 25000, 20, 18, '2026-09-25 20:00:00', '2026-09-25 21:00:00', 'AVAILABLE', '2026-09-25 10:00:00'),
(6, 3, 3, 'Cơm Tấm Cuối Ngày', 'Suất cơm tấm sườn trứng.',
    60000, 22000, 10, 9, '2026-09-23 20:00:00', '2026-09-23 21:00:00', 'EXPIRED', '2026-09-23 16:00:00');

-- Kỳ chi trả: An Phát đã được chi kỳ 21-24/09 (đơn 1, 2). Cơm Tấm 365 chưa được chi (đơn 3).
INSERT INTO payouts (id, payout_code, store_id, period_from, period_to, order_count, gross_amount,
       commission_amount, net_amount, bank_name, bank_account_no, bank_account_name, status,
       transfer_ref, note, created_by, created_at, paid_by, paid_at) VALUES
(1, 'PO-1-20260924', 1, '2026-09-21', '2026-09-24', 2, 200000, 30000, 170000,
    'Vietcombank', '0071000123456', 'NGUYEN VAN PHAT', 'PAID',
    'VCB-FT26267112233', 'Chi trả tuần 21-24/09', 1, '2026-09-25 09:00:00', 1, '2026-09-25 09:30:00');

-- Đơn hàng (phí 15%). Đơn 4 gồm 2 loại túi của Sweet Paris (đặt từ giỏ hàng).
INSERT INTO orders (id, order_code, customer_id, store_id, item_count, total_amount,
       commission_rate, commission_amount, store_earning, pickup_start, pickup_end, pickup_code,
       status, payment_status, payout_id, cancel_reason, cancelled_by, cancelled_at, handled_by,
       payment_expires_at, completed_at, created_at) VALUES
(1, 'FR260923-0001',  8, 1, 2,  80000, 15.00, 12000,  68000, '2026-09-23 20:30:00', '2026-09-23 21:30:00', 'K7Q2M9', 'COMPLETED',
'PAID',           1,    NULL, NULL, NULL, 6,    '2026-09-23 19:25:00', '2026-09-23 20:45:00', '2026-09-23 19:15:00'),
(2, 'FR260923-0002',  9, 1, 3, 120000, 15.00, 18000, 102000, '2026-09-23 20:30:00', '2026-09-23 21:30:00', 'P3X8D1', 'COMPLETED',
'PAID',           1,    NULL, NULL, NULL, 6,    '2026-09-23 19:40:00', '2026-09-23 21:10:00', '2026-09-23 19:30:00'),
(3, 'FR260923-0003', 11, 3, 1,  22000, 15.00,  3300,  18700, '2026-09-23 20:00:00', '2026-09-23 21:00:00', 'T5N6B4', 'NO_SHOW',
'PAID',           NULL, NULL, NULL, NULL, NULL, '2026-09-23 18:10:00', '2026-09-23 21:30:00', '2026-09-23 18:00:00'),
(4, 'FR260925-0004',  8, 2, 2,  87000, 15.00, 13050,  73950, '2026-09-25 21:00:00', '2026-09-25 22:00:00', 'H2R9W7', 'CONFIRMED',
'PAID',           NULL, NULL, NULL, NULL, NULL, '2026-09-25 10:30:00', NULL,                 '2026-09-25 10:20:00'),
(5, 'FR260925-0005',  9, 3, 2,  50000, 15.00,  7500,  42500, '2026-09-25 20:00:00', '2026-09-25 21:00:00', 'M8C1Z5', 'CONFIRMED',
'PAID',           NULL, NULL, NULL, NULL, NULL, '2026-09-25 11:55:00', NULL,                 '2026-09-25 11:45:00'),
(6, 'FR260925-0006', 10, 1, 2,  60000, 15.00,  9000,  51000, '2026-09-25 20:30:00', '2026-09-25 21:30:00', 'W4E7J3', 'CONFIRMED',
'PAID',           NULL, NULL, NULL, NULL, NULL, '2026-09-25 12:20:00', NULL,                 '2026-09-25 12:10:00'),
(7, 'FR260925-0007', 11, 2, 1,  45000, 15.00,  6750,  38250, '2026-09-25 21:00:00', '2026-09-25 22:00:00', 'Y6U3S8', 'CANCELLED',
'REFUND_PENDING', NULL, 'Khách tại quầy đã mua hết', 3, '2026-09-25 13:20:00', NULL, '2026-09-25 13:10:00', NULL, '2026-09-25
13:00:00'),
(8, 'FR260925-0008', 10, 2, 1,  42000, 15.00,  6300,  35700, '2026-09-25 21:00:00', '2026-09-25 22:00:00', 'G9L2V6',
'PENDING_PAYMENT', 'UNPAID',         NULL, NULL, NULL, NULL, NULL, '2026-09-25 13:40:00', NULL,                 '2026-09-25
13:30:00');

INSERT INTO order_items (order_id, batch_id, quantity, unit_price, subtotal) VALUES
(1, 2, 2, 40000,  80000),
(2, 2, 3, 40000, 120000),
(3, 6, 1, 22000,  22000),
(4, 3, 1, 45000,  45000),
(4, 4, 1, 42000,  42000),
(5, 5, 2, 25000,  50000),
(6, 1, 2, 30000,  60000),
(7, 3, 1, 45000,  45000),
(8, 4, 1, 42000,  42000);

-- Sổ kho (khớp với quantity / remaining_quantity ở trên)
INSERT INTO stock_movements (batch_id, change_qty, reason, order_id, actor_id, remaining_after, note, created_at) VALUES
(6,  10, 'LISTED',   NULL, 4,    10, 'Đăng túi',                   '2026-09-23 16:00:00'),
(2,   5, 'LISTED',   NULL, 2,     5, 'Đăng túi',                   '2026-09-23 17:00:00'),
(6,  -1, 'ORDERED',  3,    11,    9, 'Đơn FR260923-0003',          '2026-09-23 18:00:00'),
(2,  -2, 'ORDERED',  1,    8,     3, 'Đơn FR260923-0001',          '2026-09-23 19:15:00'),
(2,  -3, 'ORDERED',  2,    9,     0, 'Đơn FR260923-0002',          '2026-09-23 19:30:00'),
(3,  15, 'LISTED',   NULL, 7,    15, 'Đăng túi',                   '2026-09-25 08:00:00'),
(4,   8, 'LISTED',   NULL, 3,     8, 'Đăng túi',                   '2026-09-25 08:05:00'),
(1,   8, 'LISTED',   NULL, 6,     8, 'Đăng túi',                   '2026-09-25 09:00:00'),
(5,  20, 'LISTED',   NULL, 4,    20, 'Đăng túi',                   '2026-09-25 10:00:00'),
(3,  -1, 'ORDERED',  4,    8,    14, 'Đơn FR260925-0004',          '2026-09-25 10:20:00'),
(4,  -1, 'ORDERED',  4,    8,     7, 'Đơn FR260925-0004',          '2026-09-25 10:20:00'),
(1,   2, 'ADDED',    NULL, 6,    10, 'Làm thêm 2 túi',             '2026-09-25 11:00:00'),
(5,  -2, 'ORDERED',  5,    9,    18, 'Đơn FR260925-0005',          '2026-09-25 11:45:00'),
(1,  -2, 'ORDERED',  6,    10,    8, 'Đơn FR260925-0006',          '2026-09-25 12:10:00'),
(3,  -1, 'ORDERED',  7,    11,   13, 'Đơn FR260925-0007',          '2026-09-25 13:00:00'),
(3,   1, 'RETURNED', 7,    3,    14, 'Huỷ đơn FR260925-0007',      '2026-09-25 13:20:00'),
(4,  -1, 'ORDERED',  8,    10,    6, 'Đơn FR260925-0008',          '2026-09-25 13:30:00');

-- Giỏ hàng hiện tại: Linh đang chọn 2 túi của Sweet Paris; Bảo đang chọn 1 túi An Phát
INSERT INTO cart_items (customer_id, batch_id, quantity, added_at) VALUES
( 9, 3, 1, '2026-09-25 14:00:00'),
( 9, 4, 2, '2026-09-25 14:01:00'),
(10, 1, 1, '2026-09-25 14:05:00');

INSERT INTO payments (id, order_id, amount, txn_ref, gateway_txn_no, bank_code, response_code, status, paid_at) VALUES
(1, 1, 80000,  'FR260923-0001-1', '14567001', 'NCB', '00', 'SUCCESS', '2026-09-23 19:16:00'),
```

```
(2, 2, 120000, 'FR260923-0002-1', '14567002', 'NCB', '00', 'SUCCESS', '2026-09-23 19:31:00'),
(3, 3, 22000,  'FR260923-0003-1', '14567003', 'NCB', '00', 'SUCCESS', '2026-09-23 18:01:00'),
(4, 4, 87000,  'FR260925-0004-1', '14567004', 'NCB', '00', 'SUCCESS', '2026-09-25 10:21:00'),
(5, 5, 50000,  'FR260925-0005-1', '14567005', 'NCB', '00', 'SUCCESS', '2026-09-25 11:46:00'),
(6, 6, 60000,  'FR260925-0006-1', '14567006', 'NCB', '00', 'SUCCESS', '2026-09-25 12:11:00'),
(7, 7, 45000,  'FR260925-0007-1', '14567007', 'NCB', '00', 'SUCCESS', '2026-09-25 13:01:00'),
(8, 8, 42000,  'FR260925-0008-1', NULL,       NULL,  NULL, 'PENDING', NULL);

INSERT INTO refunds (refund_code, order_id, payment_id, amount, reason, status, requested_by, requested_at) VALUES
('RF-FR260925-0007', 7, 7, 45000, 'STORE_CANCELLED', 'PENDING', 3, '2026-09-25 13:20:00');

INSERT INTO reviews (order_id, customer_id, store_id, rating, comment, created_at) VALUES
(1, 8, 1, 5, 'Burger còn nóng, bánh mềm. Giá giải cứu quá rẻ, ủng hộ mô hình chống lãng phí!', '2026-09-23 21:40:00'),
(2, 9, 1, 4, 'Ngon, nhưng 1 cái burger hơi nguội.', '2026-09-23 21:50:00');

INSERT INTO audit_logs (actor_id, action, entity_type, entity_id, amount, detail, created_at) VALUES
(1,    'STORE_APPROVED',   'STORE',  1, NULL,   'Duyệt cửa hàng An Phát, phí 15%',                '2026-09-01 09:00:00'),
(1,    'STORE_APPROVED',   'STORE',  2, NULL,   'Duyệt cửa hàng Sweet Paris, phí 15%',            '2026-09-02 10:00:00'),
(1,    'STORE_APPROVED',   'STORE',  3, NULL,   'Duyệt cửa hàng Cơm Tấm 365, phí 15%',            '2026-09-03 14:00:00'),
(NULL, 'PAYMENT_SUCCESS',  'ORDER',  3, 22000,  'VNPay 14567003',                                 '2026-09-23 18:01:00'),
(NULL, 'PAYMENT_SUCCESS',  'ORDER',  1, 80000,  'VNPay 14567001',                                 '2026-09-23 19:16:00'),
(NULL, 'PAYMENT_SUCCESS',  'ORDER',  2, 120000, 'VNPay 14567002',                                 '2026-09-23 19:31:00'),
(6,    'ORDER_COMPLETED',  'ORDER',  1, 80000,  'Giao hàng mã K7Q2M9',                            '2026-09-23 20:45:00'),
(6,    'ORDER_COMPLETED',  'ORDER',  2, 120000, 'Giao hàng mã P3X8D1',                            '2026-09-23 21:10:00'),
(NULL, 'ORDER_NO_SHOW',    'ORDER',  3, 22000,  'Quá giờ nhận 30 phút',                           '2026-09-23 21:30:00'),
(1,    'PAYOUT_CREATED',   'PAYOUT', 1, 170000, 'Kỳ 21-24/09, 2 đơn',                             '2026-09-25 09:00:00'),
(1,    'PAYOUT_PAID',      'PAYOUT', 1, 170000, 'Chuyển khoản VCB-FT26267112233',                 '2026-09-25 09:30:00'),
(NULL, 'PAYMENT_SUCCESS',  'ORDER',  4, 87000,  'VNPay 14567004',                                 '2026-09-25 10:21:00'),
(NULL, 'PAYMENT_SUCCESS',  'ORDER',  5, 50000,  'VNPay 14567005',                                 '2026-09-25 11:46:00'),
(NULL, 'PAYMENT_SUCCESS',  'ORDER',  6, 60000,  'VNPay 14567006',                                 '2026-09-25 12:11:00'),
(NULL, 'PAYMENT_SUCCESS',  'ORDER',  7, 45000,  'VNPay 14567007',                                 '2026-09-25 13:01:00'),
(3,    'ORDER_CANCELLED',  'ORDER',  7, 45000,  'Khách tại quầy đã mua hết',                      '2026-09-25 13:20:00'),
(3,    'REFUND_REQUESTED', 'REFUND', 1, 45000,  'RF-FR260925-0007, lý do STORE_CANCELLED',        '2026-09-25 13:20:00');
```

Phụ lục B. Câu truy vấn kiểm tra dữ liệu

File đi kèm: `foodrescue_checks.sql`. Xem giải thích ở mục 5.10.

```
-- ======================================================================
--  FOODRESCUE - Kiểm tra tính đúng của dữ liệu (chạy sau khi test / trước khi demo)
--  Mỗi câu kiểm tra trả về các dòng VI PHẠM. Dữ liệu đúng => mọi câu C1..C12 trả về 0 dòng,
--  câu C0 (đối soát tổng) phải có chenh_lech = 0, câu C13 chỉ để xem số liệu từng cửa hàng.
--  Nhóm: tiền (C0, C4-C9, C12) · kho hàng (C1, C1b, C1c, C2) · đơn và giỏ hàng (C3-C3d, C5, C11) · đánh giá (C10)
-- ======================================================================
USE foodrescuedb;

-- C0. Đối soát tổng: tiền thu từ khách = hoàn + chờ hoàn + đã chi + chờ chi + nợ cửa hàng + phí + đơn chưa giao
SELECT t.*, tien_thu - (da_hoan + cho_hoan + da_chi + cho_chi + no_cua_hang + phi_nen_tang + don_chua_giao) AS chenh_lech
FROM (
  SELECT
    (SELECT COALESCE(SUM(amount), 0)            FROM payments WHERE status = 'SUCCESS')                              AS tien_thu,
    (SELECT COALESCE(SUM(amount), 0)            FROM refunds  WHERE status = 'COMPLETED')                            AS da_hoan,
    (SELECT COALESCE(SUM(amount), 0)            FROM refunds  WHERE status = 'PENDING')                              AS cho_hoan,
    (SELECT COALESCE(SUM(net_amount), 0)        FROM payouts  WHERE status = 'PAID')                                 AS da_chi,
    (SELECT COALESCE(SUM(net_amount), 0)        FROM payouts  WHERE status = 'PENDING')                              AS cho_chi,
    (SELECT COALESCE(SUM(store_earning), 0)     FROM orders   WHERE status IN ('COMPLETED','NO_SHOW') AND payout_id IS NULL) AS
no_cua_hang,
    (SELECT COALESCE(SUM(commission_amount), 0) FROM orders   WHERE status IN ('COMPLETED','NO_SHOW'))              AS
phi_nen_tang,
    (SELECT COALESCE(SUM(total_amount), 0)      FROM orders   WHERE status = 'CONFIRMED')                            AS
don_chua_giao
) t;

-- C1. Tồn kho: quantity - remaining phải bằng tổng số túi trong các đơn còn hiệu lực
SELECT b.id, b.quantity, b.remaining_quantity,
       COALESCE(SUM(CASE WHEN o.status IN ('PENDING_PAYMENT','CONFIRMED','COMPLETED','NO_SHOW') THEN i.quantity END), 0) AS
da_ban
FROM rescue_batches b
LEFT JOIN order_items i ON i.batch_id = b.id
LEFT JOIN orders o      ON o.id = i.order_id
GROUP BY b.id
HAVING b.quantity - b.remaining_quantity <> da_ban;

-- C1b. Sổ kho khớp tồn kho: tổng mọi biến động = remaining; tổng LISTED + ADDED + REMOVED = quantity;
--      dòng cuối cùng có remaining_after = remaining
SELECT b.id, b.quantity, b.remaining_quantity,
       COALESCE(SUM(m.change_qty), 0) AS tong_bien_dong,
       COALESCE(SUM(CASE WHEN m.reason IN ('LISTED','ADDED','REMOVED') THEN m.change_qty END), 0) AS tong_dua_len,
       (SELECT m2.remaining_after FROM stock_movements m2 WHERE m2.batch_id = b.id ORDER BY m2.created_at DESC, m2.id DESC LIMIT
1) AS remaining_cuoi
FROM rescue_batches b LEFT JOIN stock_movements m ON m.batch_id = b.id
GROUP BY b.id
HAVING tong_bien_dong <> b.remaining_quantity OR tong_dua_len <> b.quantity OR remaining_cuoi <> b.remaining_quantity;

-- C1c. Sổ kho khớp đơn: mỗi túi trong đơn có đúng 1 dòng ORDERED cùng số lượng; đơn đã EXPIRED/CANCELLED có dòng RETURNED
SELECT * FROM (
  SELECT i.order_id, i.batch_id, i.quantity, o.status,
         (SELECT -SUM(change_qty) FROM stock_movements m WHERE m.order_id = i.order_id AND m.batch_id = i.batch_id AND m.reason =
'ORDERED')  AS da_tru,
         (SELECT  SUM(change_qty) FROM stock_movements m WHERE m.order_id = i.order_id AND m.batch_id = i.batch_id AND m.reason =
'RETURNED') AS da_tra
  FROM order_items i JOIN orders o ON o.id = i.order_id
) x
WHERE da_tru IS NULL OR da_tru <> quantity
   OR (status IN ('EXPIRED','CANCELLED') AND (da_tra IS NULL OR da_tra <> quantity))
   OR (status NOT IN ('EXPIRED','CANCELLED') AND da_tra IS NOT NULL);

-- C2. Túi hết hàng phải ở trạng thái SOLD_OUT (hoặc đã hết hạn / huỷ)
SELECT id, remaining_quantity, status FROM rescue_batches
WHERE (remaining_quantity = 0 AND status = 'AVAILABLE') OR (remaining_quantity > 0 AND status = 'SOLD_OUT');

-- C3. Mọi túi trong đơn phải thuộc cửa hàng của đơn
SELECT o.id, o.store_id, b.id AS batch_id, b.store_id AS batch_store
FROM orders o JOIN order_items i ON i.order_id = o.id JOIN rescue_batches b ON b.id = i.batch_id
WHERE o.store_id <> b.store_id;

-- C3b. Tổng đơn = tổng chi tiết; số túi = tổng số lượng chi tiết; đơn nào cũng có ít nhất 1 dòng chi tiết
SELECT o.id, o.total_amount, SUM(i.subtotal) AS tong_chi_tiet, o.item_count, SUM(i.quantity) AS so_tui
FROM orders o LEFT JOIN order_items i ON i.order_id = o.id
GROUP BY o.id
HAVING COUNT(i.id) = 0 OR o.total_amount <> SUM(i.subtotal) OR o.item_count <> SUM(i.quantity);
```

```
-- C3c. Khung giờ lấy của đơn phải nằm trong khung giờ của mọi túi trong đơn
SELECT o.id, o.pickup_start, o.pickup_end, b.id AS batch_id, b.pickup_start AS b_start, b.pickup_end AS b_end
FROM orders o JOIN order_items i ON i.order_id = o.id JOIN rescue_batches b ON b.id = i.batch_id
WHERE o.pickup_start < b.pickup_start OR o.pickup_end > b.pickup_end;

-- C3d. Giỏ hàng: mỗi khách chỉ chứa túi của 1 cửa hàng
SELECT c.customer_id, COUNT(DISTINCT b.store_id) AS so_cua_hang
FROM cart_items c JOIN rescue_batches b ON b.id = c.batch_id
GROUP BY c.customer_id
HAVING COUNT(DISTINCT b.store_id) > 1;

-- C4. Đơn đã trả tiền (PAID / REFUND_*) phải có đúng 1 giao dịch SUCCESS bằng đúng số tiền; đơn UNPAID thì không có
SELECT o.id, o.payment_status, o.total_amount, COUNT(p.id) AS so_gd_thanh_cong, COALESCE(SUM(p.amount), 0) AS tong_gd
FROM orders o LEFT JOIN payments p ON p.order_id = o.id AND p.status = 'SUCCESS'
GROUP BY o.id
HAVING (o.payment_status <> 'UNPAID' AND (COUNT(p.id) <> 1 OR SUM(p.amount) <> o.total_amount))
    OR (o.payment_status = 'UNPAID' AND COUNT(p.id) > 0);

-- C5. Trạng thái thanh toán phải khớp trạng thái đơn
SELECT id, status, payment_status FROM orders
WHERE (status IN ('CONFIRMED','COMPLETED','NO_SHOW') AND payment_status <> 'PAID')
   OR (status IN ('PENDING_PAYMENT','EXPIRED')        AND payment_status <> 'UNPAID')
   OR (status = 'CANCELLED' AND payment_status = 'PAID');

-- C6. Đơn REFUND_PENDING / REFUNDED phải có đúng 1 bản ghi hoàn tiền tương ứng, số tiền bằng total_amount
SELECT o.id, o.payment_status, r.status AS refund_status, o.total_amount, r.amount
FROM orders o LEFT JOIN refunds r ON r.order_id = o.id
WHERE (o.payment_status = 'REFUND_PENDING' AND (r.id IS NULL OR r.status <> 'PENDING'   OR r.amount <> o.total_amount))
   OR (o.payment_status = 'REFUNDED'       AND (r.id IS NULL OR r.status <> 'COMPLETED' OR r.amount <> o.total_amount))
   OR (o.payment_status IN ('UNPAID','PAID') AND r.id IS NOT NULL);

-- C7. Kỳ chi trả: số đơn và số tiền phải bằng tổng của đúng các đơn gắn vào kỳ; đơn phải cùng cửa hàng
SELECT p.id, p.order_count, COUNT(o.id) AS so_don, p.gross_amount, SUM(o.total_amount) AS gross,
       p.commission_amount, SUM(o.commission_amount) AS phi, p.net_amount, SUM(o.store_earning) AS net
FROM payouts p LEFT JOIN orders o ON o.payout_id = p.id
WHERE p.status <> 'CANCELLED'
GROUP BY p.id
HAVING p.order_count <> COUNT(o.id) OR p.gross_amount <> SUM(o.total_amount)
    OR p.commission_amount <> SUM(o.commission_amount) OR p.net_amount <> SUM(o.store_earning)
    OR SUM(o.store_id <> p.store_id) > 0;

-- C8. Kỳ chi trả đã huỷ không được còn đơn gắn vào
SELECT p.id, COUNT(o.id) FROM payouts p JOIN orders o ON o.payout_id = p.id WHERE p.status = 'CANCELLED' GROUP BY p.id;

-- C9. Đơn trong kỳ chi trả phải có ngày chốt đơn nằm trong kỳ
SELECT o.id, o.completed_at, p.period_from, p.period_to FROM orders o JOIN payouts p ON p.id = o.payout_id
WHERE DATE(o.completed_at) NOT BETWEEN p.period_from AND p.period_to;

-- C10. Đánh giá phải thuộc đơn COMPLETED, đúng khách và đúng cửa hàng của đơn
SELECT r.id FROM reviews r JOIN orders o ON o.id = r.order_id
WHERE o.status <> 'COMPLETED' OR r.customer_id <> o.customer_id OR r.store_id <> o.store_id;

-- C11. Người giao hàng phải là chủ hoặc nhân viên của đúng cửa hàng
SELECT o.id, o.handled_by FROM orders o
JOIN users u ON u.id = o.handled_by
JOIN stores s ON s.id = o.store_id
WHERE NOT ((u.role = 'STORE_OWNER' AND s.owner_id = u.id) OR (u.role = 'STORE_STAFF' AND u.store_id = s.id));

-- C12. Mọi giao dịch tiền quan trọng phải có dòng nhật ký
SELECT 'PAYMENT_SUCCESS' AS thieu_log, p.order_id AS id FROM payments p
 WHERE p.status = 'SUCCESS'
   AND NOT EXISTS (SELECT 1 FROM audit_logs a WHERE a.action = 'PAYMENT_SUCCESS' AND a.entity_type = 'ORDER' AND a.entity_id =
p.order_id)
UNION ALL
SELECT 'REFUND_REQUESTED', r.id FROM refunds r
 WHERE NOT EXISTS (SELECT 1 FROM audit_logs a WHERE a.action = 'REFUND_REQUESTED' AND a.entity_type = 'REFUND' AND a.entity_id =
r.id)
UNION ALL
SELECT 'PAYOUT_PAID', p.id FROM payouts p
 WHERE p.status = 'PAID'
   AND NOT EXISTS (SELECT 1 FROM audit_logs a WHERE a.action = 'PAYOUT_PAID' AND a.entity_type = 'PAYOUT' AND a.entity_id =
p.id);

-- C13. Điểm đánh giá và doanh thu từng cửa hàng (xem, không phải kiểm tra lỗi)
SELECT s.id, s.name,
       (SELECT ROUND(AVG(rating), 2) FROM reviews r WHERE r.store_id = s.id)                                    AS diem_tb,
       (SELECT COALESCE(SUM(total_amount), 0)      FROM orders o WHERE o.store_id = s.id AND o.status IN ('COMPLETED','NO_SHOW'))
AS doanh_thu,
       (SELECT COALESCE(SUM(commission_amount), 0) FROM orders o WHERE o.store_id = s.id AND o.status IN ('COMPLETED','NO_SHOW'))
AS phi,
       (SELECT COALESCE(SUM(store_earning), 0)     FROM orders o WHERE o.store_id = s.id AND o.status IN ('COMPLETED','NO_SHOW'))
```

```
AS thuc_nhan,
       (SELECT COALESCE(SUM(net_amount), 0)        FROM payouts p WHERE p.store_id = s.id AND p.status = 'PAID')
AS da_chi,
       (SELECT COALESCE(SUM(store_earning), 0)     FROM orders o WHERE o.store_id = s.id AND o.status IN ('COMPLETED','NO_SHOW')
AND o.payout_id IS NULL) AS con_no
FROM stores s;
```